k-NN, Naive Bayes and Support Vector Machines — Commentary

Lesson 6 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

Not examinable · reading time about 2 hours

Contents

Slide 1 — k-NN, Naive Bayes and Support Vector Machines . . . 4

Part I — Three philosophies, one dataset **4**
Slide 2 — Before we start . . . 4
Slide 3 — Today: the first lesson that offers a choice . . . 4
Slide 4 — 1,200 pumps, two readings each . . . 5
Slide 5 — The shape of the problem: healthy pumps in the middle, faulty all around . . . 5
Slide 6 — What a straight boundary costs . . . 6

Part II — k-nearest neighbours **6**
Slide 7 — k-nearest neighbours: the whole algorithm . . . 6
Worked example — five stored pumps, three values of k . . . 7
Slide 8 — k-NN, drawn: one new pump and its five nearest . . . 7
Slide 9 — "Closest" means the smallest Euclidean distance . . . 8
Slide 10 — Scale first, or vibration decides everything . . . 8
Worked example — who is nearest depends on the units . . . 9
Slide 11 — k sets the balance between bias and variance . . . 9
Slide 12 — Training against cross-validated accuracy, by k (optional) . . . 9
Slide 13 — Accuracy against k, between the baseline and the noise ceiling . . . 10
Slide 14 — The boundary k-NN draws at k = 1, 15 and 401 . . . 11
Slide 15 — What k-NN costs . . . 11

Part III — The curse of dimensionality **11**
Slide 16 — An experiment: add columns of pure noise . . . 11
Slide 17 — The experiment, drawn . . . 12
Slide 18 — The two real readings are still there, and accuracy still falls . . . 12
Slide 19 — Why: distances stop varying . . . 13
Slide 20 — The nearest point stops being near . . . 13
Worked example — how big a neighbourhood holds 1% of the data . . . 14
Slide 21 — Nearest divided by farthest (optional) . . . 14
Slide 22 — In 100 dimensions the nearest point is 70% as far as the farthest . . . 14
Slide 23 — What the curse is, and is not . . . 15
Slide 24 — Notebook 1, live . . . 15
Slide 25 — Break . . . 15

Part IV — Naive Bayes **15**
Slide 26 — Naive Bayes: turn the question around . . . 15
Slide 27 — Bayes' rule, applied to a class . . . 16
Slide 28 — Drop the denominator: it is the same for every class . . . 16
Worked example — one pump, two classes, with and without the denominator . . . 16
Slide 29 — The assumption: independent, given the class . . . 16
Question — what does "independent given the class" mean, and why did we assume it for the pumps? . . . 17
Question — an example of features that really are independent given the class . . . 17
Slide 30 — The assumption, written: a product of n densities . . . 18
Worked example — what the assumption saves, counted . . . 18
Slide 31 — Gaussian: each factor is a bell curve (optional) . . . 18
Slide 32 — Training computes ten numbers: the widths separate the classes . . . 19
Slide 33 — A pump at 48 Hz, 5.6 bar: there the faulty bell is 86 times higher . . . 19
Slide 34 — Take logs: the product becomes a sum (optional) . . . 20
Slide 35 — 48 Hz, 5.6 bar: scores and their logs (optional) . . . 20
Slide 36 — From the two totals to probabilities (optional) . . . 20
Slide 37 — Software uses only the gap between the logs (optional) . . . 21
Slide 38 — A cheap model, built on an assumption that is almost never true . . . 21
Slide 39 — Naive Bayes on the pumps: 0.933 . . . 21
Question — can a good accuracy tell me that the assumption was true? . . . 22
Slide 40 — Uncorrelated within each class, but not independent . . . 22
Question — what does the correlation code do? . . . 23
Question — what do the squared-distance results say, and where do 0.784 and 1.309 come from? . . . 23
Question — how would you check the assumption with many features? . . . 24
Slide 41 — The densities are wrong, but the boundary they draw is about right . . . 25
Slide 42 — A second dataset: the signal is an interaction . . . 25
Slide 43 — Each sensor alone: the two classes look the same . . . 26
Question — what do the interacting sensors' numbers show? . . . 26
Worked example — the interaction, counted . . . 27
Slide 44 — 0.404: below the baseline, and below chance . . . 27
Slide 45 — Its probabilities cannot be trusted . . . 28
Question — is this the same distinction as the area under the curve against calibration? 28
Question — what does section 5 of notebook 2 mean? . . . 28
Slide 46 — Notebook 2, live . . . 29
Question — what do the notebook's conclusions mean? . . . 29

Part V — Support vector machines **30**
Slide 47 — The margin: choosing among lines that all fit . . . 30
Question — why does logistic regression with gradient descent not find the widest margin? . . . 30
Worked example — distance to a boundary, and the width of the slab . . . 31
Slide 48 — 80 points in two groups: three of them fix the line (support vectors) . . . 31
Slide 49 — Not separable? Charge for the violations . . . 32
Slide 50 — The hinge loss stops at zero (optional) . . . 32
Worked example — five pumps, two losses . . . 33
Slide 51 — The SVM objective: small w, plus C times the slack (optional) . . . 33

Question — is training the minimisation of the hinge loss, and what is ξ_i a function of? . . . 33
Question — so some points are accepted inside the margin, or even on the wrong side? 34
Slide 52 — C is the price of a training error . . . 34
Question — in what sense is the margin "deformed"? Is it not always straight? . . . 34
Worked example — two candidate boundaries, three prices . . . 35
Slide 53 — The best straight line is still a straight line . . . 35
Slide 54 — Many support vectors: a free warning . . . 35
Slide 55 — A new coordinate makes the classes separable . . . 35
Slide 56 — The same pumps, lifted . . . 36
Slide 57 — The SVM's solution: a weighted sum of pumps (optional) . . . 36
Question — what is the dual formulation, intuitively? . . . 37
Slide 58 — So the new coordinates are never computed . . . 37
Slide 59 — The kernel trick by hand: 25 = 25 (optional) . . . 37
Question — how do I find $K(x_i, x_j)$ without knowing $\phi(x_i) \cdot \phi(x_j)$? . . . 38
Slide 60 — The RBF kernel: similarity falls with distance . . . 38
Question — what does RBF stand for? . . . 39
Worked example — how similar two pumps are, by distance . . . 39
Slide 61 — Training: one weight per pump (optional) . . . 39
Question — who are x_i and x, during training and during prediction? . . . 40
Slide 62 — The SVM on 48 Hz, 5.6 bar: +1.883 (optional) . . . 40
Slide 63 — One pump's vote, drawn . . . 41
Slide 64 — Training against cross-validated, three settings (optional) . . . 41
Slide 65 — How far one support vector's vote reaches, for four values of . . . 42
Slide 66 — Overfitting you can see: the boundary at the three settings . . . 42

Part VI — Closing **43**
Slide 67 — Notebook 3, live . . . 43
Slide 68 — The three families compared, on the same pumps and the same folds . . . 43
Slide 69 — How to choose, in practice . . . 43
Slide 70 — What to take away . . . 44
Slide 71 — Homework . . . 44

What this document is. A companion to the slides of lesson 6: for every slide, what it means, the steps it leaves implicit, and the questions it tends to raise, answered under the slide they belong to. It is a second explanation of the same material, in a different voice from the handout, which remains the reference for the derivations. It is **not examinable**: nothing here is needed for the exam that is not already in the handout and the notebooks.

Slide numbers and titles are the ones in the lesson's slide deck, in `Slides/`. Where a section mentions "the notes", it means the speaker notes stored in the slide deck, which PowerPoint shows below each slide.

Worked examples sit under ten slides (7, 10, 20, 28, 30, 43, 47, 50, 52, 60): a handful of invented values each, small enough to work through by hand. Their numbers are not the lesson's data; every one is computed by `commentary_examples.py`, beside this file, which also writes them here.

Slide 1 — k-NN, Naive Bayes and Support Vector Machines

The title slide, but it announces the idea that runs through the whole lesson: these are not just three more classifiers, they are three different ways of thinking about what a model is.

- **k-nearest neighbours (k-NN)** reasons from examples. It keeps the training data and, when a new point arrives, looks at what happens near it.
- **Naive Bayes** does almost the opposite. It compresses the data into a small probabilistic model, at the price of a strong assumption: the features are independent once the class is known.
- **The support vector machine (SVM)** reasons geometrically. It looks for the boundary that leaves the **widest margin** around it and, when a straight boundary is not enough, changes the space the data lives in, through a kernel.

In one line: **k-NN remembers, Naive Bayes assumes, the SVM separates.**

Part I — Three philosophies, one dataset

Slide 2 — Before we start

The slide links lesson 6 to lesson 5. Lesson 5 built the tools for judging whether a result can be believed: cross-validation, baselines, leakage, the comparison between training and validation scores, bias and variance. Changing model does not retire them. Lesson 6 is where **model selection** begins in earnest: knowing how to train a model is not enough; you have to decide which *family* of models suits the structure of the problem. The opening discussion of Exercise 5 is a reminder that a high number does not make a good model: that exercise hid more than one problem on purpose, and the task was to diagnose why the first result was too good.

Slide 3 — Today: the first lesson that offers a choice

Until now the course has followed one path: data, preprocessing, a linear model, optimisation, evaluation. Now a new question appears: **which family of models should I choose?** The three answers of the day are:

- **k-NN — remember and vote.** It learns almost nothing explicitly; it keeps the examples.
- **Naive Bayes — make a strong assumption.** It assumes the features are independent given the class, and gets in return a model that is very easy to estimate.
- **SVM — margins and kernels.** It looks for a robust separation, and can change the feature space without ever writing the new features down.

The distinction matters more than the small differences between their accuracies, and it is one of the lesson's closing messages: **choosing the right family can count far more than fine-tuning the wrong one.**

Slide 4 — 1,200 pumps, two readings each

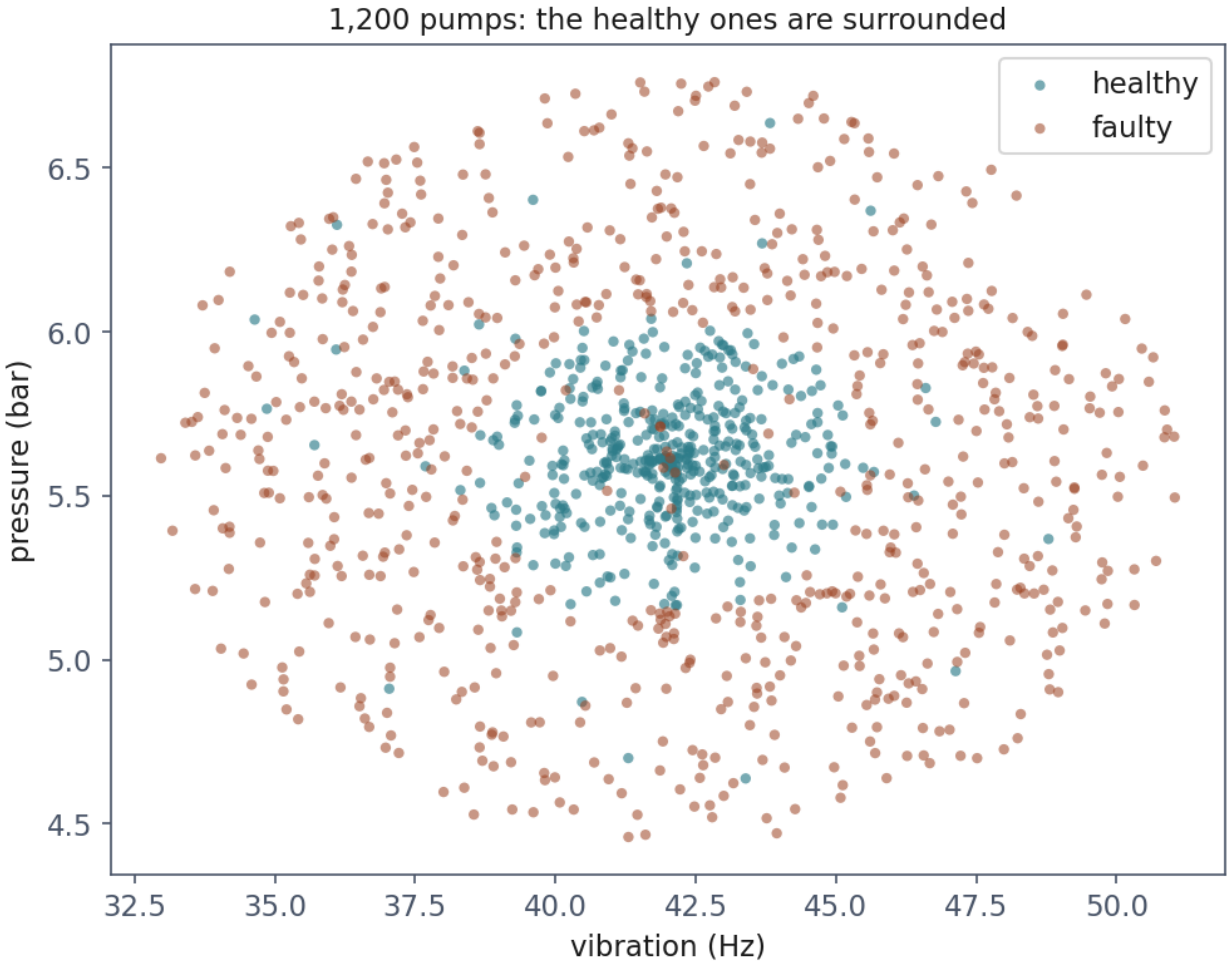
The dataset for most of the lesson: **1,200 industrial pumps**, two features each, vibration in hertz (Hz) and pressure in bar. A pump is healthy when it runs close to its design point, and faulty when it drifts too far from it **in any direction**. So the rule is not of the kind "faulty if the vibration is above a threshold": a vibration can be a problem because it is too high *or* too low, and the same holds for the pressure.

The data generator builds the problem from a standardised distance from the design point,

$$r = \sqrt{\left(\frac{x_{\text{vib}} - 42}{3.5}\right)^2 + \left(\frac{x_{\text{press}} - 5.6}{0.45}\right)^2},$$

and calls a pump faulty when $r > 1$; then it flips **4% of the labels** at random, as label noise. About **61.3%** of the pumps are faulty, so a classifier that always says "faulty" already scores 0.613. That baseline is the yardstick for every result that follows.

Slide 5 — The shape of the problem: healthy pumps in the middle, faulty all around



1,200 pumps. The healthy ones are surrounded, because a pump can fail by running too slow as well as too fast. No straight line separates these classes.

In the plane of vibration and pressure, the healthy pumps occupy a central region and the faulty ones surround it. In standardised coordinates the healthy region is roughly a **disc**; in the original units it is an **ellipse**, because the two features have different scales.

A linear boundary in two dimensions, $w_1 x_1 + w_2 x_2 + b = 0$, is a **straight line**, and a line splits the plane into two half-planes, $w^\top x + b > 0$ on one side and $w^\top x + b < 0$ on the other. What this problem needs is "inside is healthy, outside is faulty", in every direction, and no single line can enclose a central region. So the problem is not "find a better line": **no line can represent this structure at all.** That anticipates an idea that returns with the SVM: if the family of models (the *hypothesis class*) is wrong, a better optimiser cannot help.

Slide 6 — What a straight boundary costs

Model	Cross-validated accuracy
Majority baseline	0.613
Logistic regression	0.613 ± 0.000

Logistic regression is not doing "badly": it scores **exactly the baseline**, because in effect it predicts "faulty" for every pump. Note the ± 0.000. It might look like a virtue, a perfectly stable model, but the stability comes from giving the same trivial answer in every fold. A very small standard deviation does not mean a good model; it can also mean a model so rigid that it fails the same way every time.

Part II — k-nearest neighbours

Slide 7 — k-nearest neighbours: the whole algorithm

The whole of k-NN fits in one sentence: to classify a new point, find the k training examples closest to it and take a majority vote. With $k = 5$ and neighbours labelled H, H, F, H, F, there are three H against two F, so the prediction is healthy.

There is **no real training**. Logistic regression learned coefficients $w_1, \ldots, w_n, b$; the `fit()` of k-NN essentially stores the training set. That is why it is called a **lazy learner**, and the work it does not do during training is paid for at prediction time. In scikit-learn the whole algorithm is one class, `KNeighborsClassifier(n_neighbors=5)`, and its `fit()` stores the data.

Worked example — five stored pumps, three values of k

Distances from a new pump to the five stored ones (standardised units):

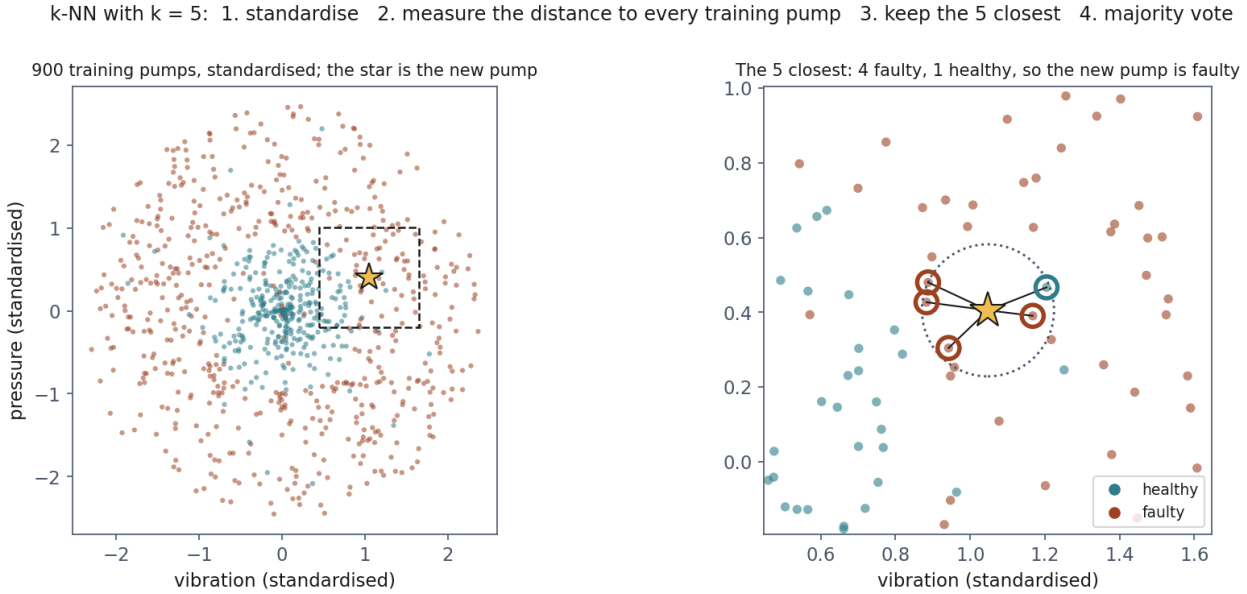
pump	distance	label
A	0.4	faulty
B	0.7	healthy
C	0.9	healthy
D	1.3	faulty
E	1.6	faulty

- $k = 1$: the nearest, A (faulty) → **faulty**
- $k = 3$: the 3 nearest, A (faulty), B (healthy), C (healthy) → **healthy**
- $k = 5$: the 5 nearest, A (faulty), B (healthy), C (healthy), D (faulty), E (faulty) → **faulty**

Nothing was fitted and nothing is estimated: the answer is a vote, and k decides who gets one. The same pump changes class twice as k grows — which is why k is chosen by cross-validation and not by taste.

Slide 8 — k-NN, drawn: one new pump and its five nearest

The previous slide stated the whole algorithm in one sentence; this one does it once, on the pumps, so the four steps can be followed with the eye. The data are notebook 1's split: 900 training pumps, the other 300 held out, both readings standardised by a scaler fitted on the 900 alone. The new pump sits at 46 Hz and 5.8 bar, the gold star.



Left, every training pump and the box the right panel enlarges. Right, the five closest pumps, circled in their class colour; the dotted circle reaches the fifth.

Read the right panel as the four steps of the strip at the top. The distances are measured in

standardised units, which is why the circle is round. Four of the five neighbours are faulty and one is healthy, so the vote is four to one and the new pump is classified faulty.

Why does the healthy neighbour not matter? Because it is outvoted. With $k = 1$ the verdict would rest on the single closest pump, and if that pump carried one of the 4% flipped labels the verdict would be wrong. A vote over five neighbours lets one odd label be outvoted, which is the first hint of why k is a trade-off.

Slide 9 — "Closest" means the smallest Euclidean distance

The Euclidean distance,

$$d(x, x') = \sqrt{\sum_{j=1}^{n} (x_j - x'_j)^2},$$

matters more than it looks. k-NN has no coefficients with which to say "vibration matters, pressure a little less, feature 17 not at all": every feature enters the distance with the same weight. A linear model can learn $w_1 = 4$ and $w_2 = 0.001$ and so learn that x_2 hardly counts; k-NN has no such mechanism. Hence the notes' phrase: **the distance function is the model.** If the distance does not express real similarity between examples, k-NN has nothing else with which to correct it.

Slide 10 — Scale first, or vibration decides everything

Suppose pump A differs from a query by 10 Hz in vibration and nothing in pressure, so $d_A = \sqrt{10^2 + 0^2} = 10$, and pump B has the same vibration and differs by 1 bar, so $d_B = 1$. k-NN concludes that B is ten times closer, but only because of the units: measure pressure in millibar and the same difference becomes 1,000, and suddenly pressure dominates.

The fix is to standardise, $z_j = (x_j - \mu_j)/\sigma_j$, so that a difference of 1 means one standard deviation on every feature. And the scaler goes **inside the pipeline**, `Pipeline([("scaler", StandardScaler()), ("knn", KNeighborsClassifier())])`, so that during cross-validation μ and σ are computed on the training rows of each fold only. Otherwise the scaling leaks.

Worked example — who is nearest depends on the units

A new pump at 42 Hz, 5.6 bar. Two stored pumps: A at 45 Hz, 5.6 bar; B at 42.5 Hz, 6.3 bar. Suppose vibration typically spreads by about 6 Hz and pressure by about 0.3 bar.

distance to	A	B	nearest
raw units	3.00	0.86	**B**
standardised	0.50	2.33	**A**

Raw, A is $\sqrt{3^2+0^2}$ away and B $\sqrt{0.5^2+0.7^2}$; standardised, A is $\sqrt{(3/6)^2+0^2}$ and B $\sqrt{(0.5/6)^2+(0.7/0.3)^2}$.

In raw units 0.7 bar looks smaller than 3 Hz, so B wins — although 0.7 bar is more than two typical spreads of pressure and 3 Hz is half a spread of vibration. Standardised, each reading is measured against its own spread and A is the real neighbour.

Slide 11 — k sets the balance between bias and variance

k is the most intuitive example of lesson 5's bias-variance trade-off.

Small k: the model looks at a very local neighbourhood and follows the details of the data closely, so low bias and high variance. In the extreme, $k = 1$: every training point is its own nearest neighbour, at distance zero, so it predicts its own label and the training accuracy is 1. That does not mean the model generalises; it means it **memorises** the training set.

Large k: the prediction averages over a growing area, small irregularities are smoothed away, variance falls and bias rises. When k becomes huge, the idea of a *local* neighbourhood disappears.

One precision from the material: "with $k = 1$ the training accuracy is always 1" assumes there are no identical points with different labels, and depends on how ties are broken. For this dataset it holds.

Slide 12 — Training against cross-validated accuracy, by k (optional)

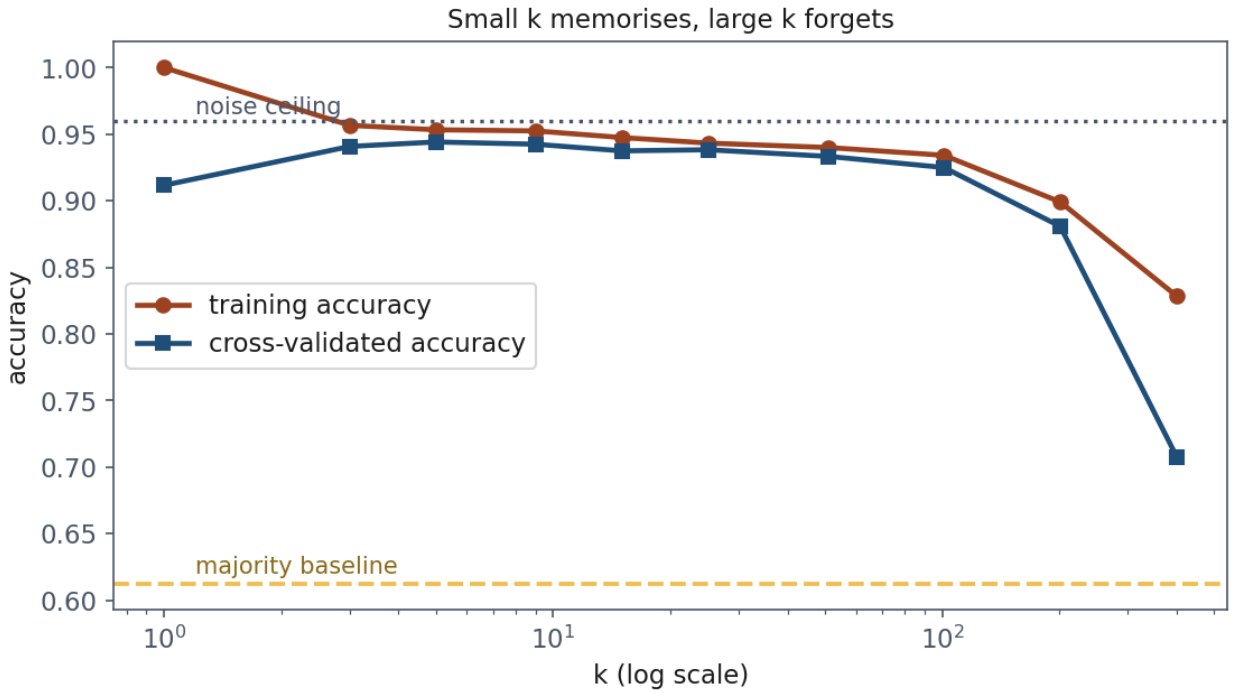
k	Training	Cross-validation
1	1.000	0.912
5	0.953	**0.944**
15	0.948	0.938
51	0.940	0.933
401	0.828	0.708

At $k = 1$, 1.000 against 0.912 is high variance: the model memorises the 4% of labels that were deliberately flipped. At $k = 5$ the neighbourhood is large enough to outvote many noisy points and still small enough to follow the real shape: 0.944. At $k = 401$, training 0.828 and validation 0.708:

this is not overfitting, because **the training score is low too**. It is the other side of the trade-off, underfitting, or high bias.

Note also that there is no single magic k: between 5, 15 and 51 the change is gradual. Many hyperparameters have a reasonably good *region*, not one narrow optimum.

Slide 13 — Accuracy against k, between the baseline and the noise ceiling

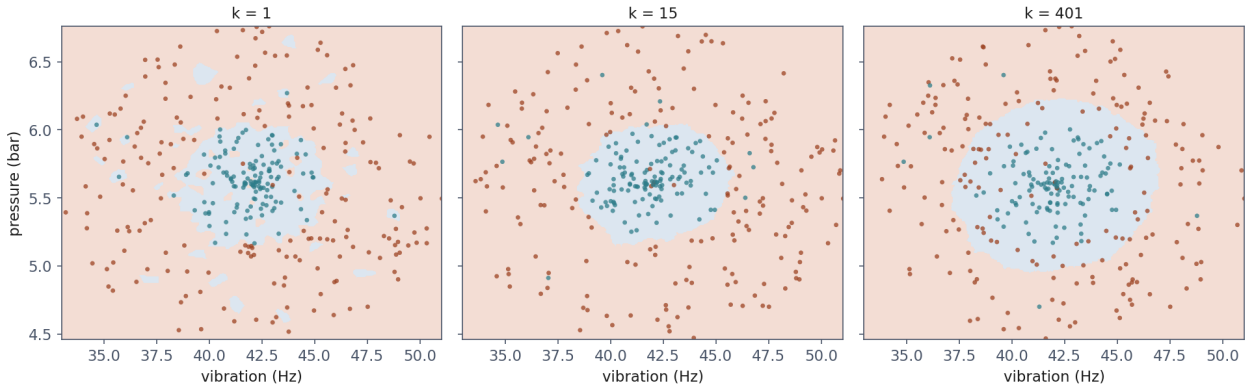


The two curves and the two lines that bound them: the noise ceiling at 0.96, which nothing can exceed, and the majority baseline at 0.613, which everything should. Note where the training curve starts.

The training accuracy starts at 1.0 at $k = 1$ and tends to fall as k grows. The validation accuracy starts lower, rises, peaks and falls again: the classic bias-variance shape. The two horizontal lines matter as much as the curves. The **noise ceiling**, about 0.96: with 4% of the labels flipped, nobody should expect 100%. The **majority baseline**, 0.613: below it, a model is worse than answering "faulty" every time.

One subtlety from the notes: the gap between training and validation is **not, in mathematical terms, "the variance"**. It is better to say that a large gap is a symptom consistent with high variance, while training and validation scores that are both low and close together are typical of high bias.

Slide 14 — The boundary k-NN draws at k = 1, 15 and 401



*The same data at three values of k. At k = 1 the boundary is ragged, with islands around individual mislabelled points. At k = 15 it is a clean disc, close to the envelope that generated the data. At k = 401 it has **inflated** past the true envelope and swallows faulty pumps — it has not collapsed to one class, it has stopped following the boundary and started averaging over it.*

The same idea, drawn. At $k = 1$ the boundary is jagged: every mislabelled point can create a small island around itself, which is variance made visible. At $k = 15$ the central region is smooth and follows the true operating envelope well. At $k = 401$ the neighbourhood is no longer local: the healthy region spreads too far and swallows faulty points. The notes stress that at $k = 401$ the model has **not** simply collapsed to the majority class; it is averaging across the true boundary until the boundary is distorted. More k means more smoothing, and too much smoothing loses the real structure.

Slide 15 — What k-NN costs

So far k-NN looks almost too good: no training at all. But the cost has only moved to prediction. With m training examples and n features, a naive query computes the distance to all m points, each costing $O(n)$, so $O(mn)$ per prediction. With 1,200 pumps that is nothing; with $m = 10^7$ and thousands of queries a second it is a real problem. Data structures such as k-d trees and ball trees, and approximate nearest-neighbour methods, help, but high dimensionality often reduces what they gain.

There is a second cost: **the dataset is the model.** To deploy k-NN you deploy the training examples, with consequences for memory, latency, privacy and possibly the law. A deployed logistic regression, by contrast, is a vector w and an intercept b.

Part III — The curse of dimensionality

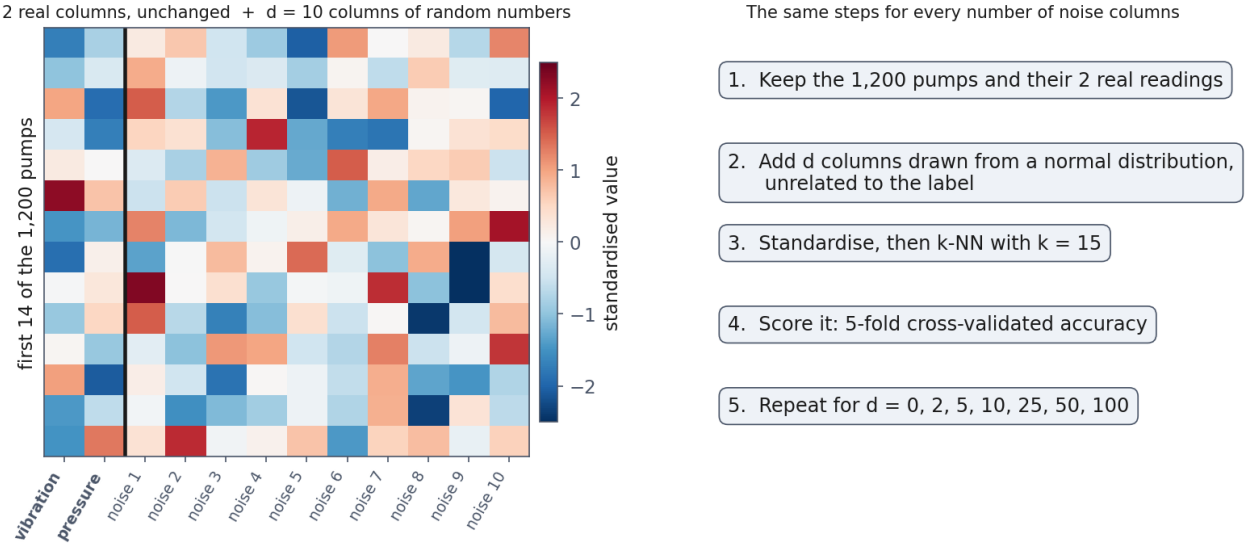
Slide 16 — An experiment: add columns of pure noise

The experiment: keep the two informative features, vibration and pressure, and append columns $z_1, \ldots, z_p$ of pure noise, $z_j \sim \mathcal{N}(0, 1)$. The function `load_with_noise()` leaves the two original columns **exactly as they were** and only adds independent Gaussian columns. So **no useful**

information is removed: the whole problem is still contained in vibration and pressure. The intuition says "a feature that does not help will be ignored". But k-NN **cannot ignore it**, because every column enters the distance.

Slide 17 — The experiment, drawn

The previous slide described the experiment in words; this one draws it, before the result. Nothing in it is a finding yet: it shows what k-NN is given, and what is repeated.



Left, the table k-NN receives with ten noise columns: the first 14 of the 1,200 pumps, every column standardised, colour for the value. Right, the five steps, repeated for every number of noise columns.

The two columns left of the black line are vibration and pressure, the same in every run. Everything to the right is drawn from a normal distribution and has nothing to do with the label: look along any noise column and the colours are as mixed for faulty pumps as for healthy ones. The model is the same each time, standardised k-NN with $k = 15$, scored by 5-fold cross-validation, so the only thing that changes from one run to the next is how many random columns sit beside the real ones.

Why draw it at all? Because the result on the next slide only means something if nothing was taken away. The picture makes that visible: the informative columns are still there, unchanged, in every run.

Slide 18 — The two real readings are still there, and accuracy still falls

Total columns	Accuracy
2	0.938
12	0.762
27	0.662
52	0.602
102	0.578

Total columns	Accuracy

With two features, 0.938. Ten useless columns bring it to 0.762; fifty, to 0.602, **below the majority baseline** of 0.613, although vibration and pressure are still there unchanged. The problem is not that the signal was lost: **the geometry k-NN used to find the signal was destroyed.** Ten useless columns cost about 18 points of accuracy, and fifty push the classifier below the baseline.

Slide 19 — Why: distances stop varying

Write the squared distance as a sum, $d^2(x, x') = \sum_{j=1}^{n} Z_j$ with $Z_j = (x_j - x'_j)^2$. If the coordinates are independent and similarly distributed, the Z_j have a common mean μ and variance v, so

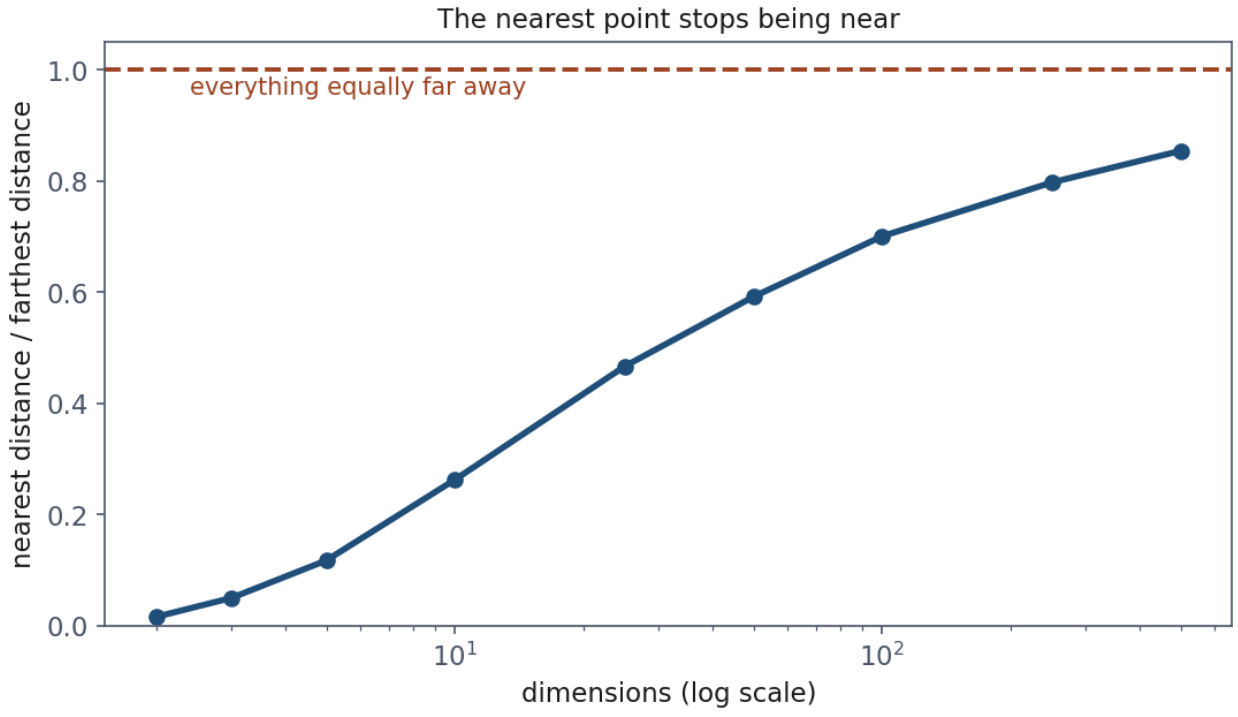
$$\mathbb{E}[d^2] = n\mu, \qquad \mathrm{Var}(d^2) = nv, \qquad \mathrm{SD}(d^2) = \sqrt{nv}.$$

The mean grows like n but the spread only like $\sqrt{n}$, so the **relative spread**

$$\frac{\mathrm{SD}(d^2)}{\mathbb{E}[d^2]} = \frac{\sqrt{nv}}{n\mu} = \frac{1}{\sqrt{n}}\,\frac{\sqrt{v}}{\mu}$$

goes to zero as n grows. That is the mathematical heart of the effect. It does not mean that distances go to zero; they may well grow. It means they become **more and more alike relative to their size**.

Slide 20 — The nearest point stops being near



As dimensions grow, the nearest point stops being near. The ratio climbs towards 1, where every point is the same distance from every other.

The slide turns the derivation into a quantity you can picture, $d_{\min}/d_{\max}$, the distance to the nearest point divided by the distance to the farthest. In a few dimensions $d_{\min} \ll d_{\max}$, so "the nearest neighbour" means something strong. In high dimension $d_{\min} \approx d_{\max}$: the nearest neighbour still exists mathematically, but it is not much closer than anything else. And the effect does not wait for thousands of features: much of it is already visible in the first few tens of dimensions.

Worked example — how big a neighbourhood holds 1% of the data

Points spread evenly in a cube of side 1. A smaller cube that contains 1% of them must have volume 0.01, so its side is $0.01^{1/d}$:

dimensions d	side of the 1% cube	as a fraction of each axis
1	$0.01^{1/1}$	**0.010**
2	$0.01^{1/2}$	**0.100**
10	$0.01^{1/10}$	**0.631**
100	$0.01^{1/100}$	**0.955**

In two dimensions the 1% neighbourhood spans a tenth of each axis — local. In ten it spans 63% of every axis, and in a hundred nearly all of it. A "nearest neighbour" found that way is not near in any useful sense; it is just the least far.

Slide 21 — Nearest divided by farthest (optional)

Dimensions	$d_{\min}/d_{\max}$
2	0.016
10	0.263
50	0.592
100	0.701
500	0.855

In two dimensions the nearest point is very close compared with the farthest, 0.016. In 100, the nearest point is already 70% as far away as the farthest; in 500, 0.855. "Near" and "far" have become hard to tell apart.

Slide 22 — In 100 dimensions the nearest point is 70% as far as the farthest

k-NN rests on an implicit assumption: the points nearest the query are especially relevant to it. If the nearest point is about as far as a random one, picking the five nearest neighbours starts to look like picking five points almost at random, and a vote among random points drifts towards the overall class proportions. In this dataset that is $P(\text{faulty}) \approx 0.613$, which is why the accuracy falls towards the **majority baseline**, not towards 0.5.

Slide 23 — What the curse is, and is not

The curse of dimensionality does **not** mean "machine learning does not work in high dimension": many models handle thousands or millions of features. The specific problem is that **distances become less discriminating as the dimension grows**, so what suffers is methods that rely directly on distances: k-NN, k-means, the radial basis function kernel. A linear model can switch off a useless feature with a coefficient near zero, $w_j \approx 0$; in k-NN the feature stays in $\sum_j (x_j - x'_j)^2$, and no coefficient removes it. Hence the value of **feature selection** — which, as lesson 5 insisted, must itself happen **inside the cross-validation pipeline**, or it leaks.

Slide 24 — Notebook 1, live

The slide adds no theory; it is where the theory is checked by experiment. Its table pairs each section of the notebook with the slides it comes from and the scikit-learn calls it uses, so that a cell you are stuck on leads you back to the idea. The notebook shows, in order, the straight line failing, k-NN without and with scaling, the effect of different values of k, the noise columns, and the geometry degrading with the dimension. The point is to see that the claims above are not just geometry: they move the accuracy measurably — 0.944 at $k = 5$, and below the baseline with fifty noise columns.

Slide 25 — Break

The break also marks a sharp turn. k-NN keeps practically the whole dataset; the next model does almost the opposite, and compresses the training set into a handful of numbers.

Part IV — Naive Bayes

Slide 26 — Naive Bayes: turn the question around

What we want is $P(y = c \mid x)$: the probability of the class given the readings of this pump. The data does not hand us that quantity directly. Bayes' rule trades it for three quantities that the data might supply:

- the **prior** $P(y = c)$, how common the class is, which is just a count;
- the **denominator** $P(x)$, which is the same for every class and so cannot change which class wins;
- the **likelihood** $P(x \mid y = c)$: how probable this exact combination of readings is among pumps of that class.

The third is where all the difficulty sits. With two features it is a two-dimensional density, and 1,200 rows can estimate it. With twenty features it is a twenty-dimensional density, and no realistic amount of data fills a twenty-dimensional space. This is the curse of dimensionality again, arriving from a different direction: before, it broke distances; here, it breaks density estimation.

Slide 27 — Bayes' rule, applied to a class

$$P(y = c \mid x) = \frac{P(x \mid y = c)\, P(y = c)}{P(x)}$$

Read the left side as "what we want" and the right side as "what the data can give": how often pumps of class c produce readings like these, times how common class c is, divided by how common readings like these are overall. The denominator does not contain c, so it is the same number whether we are scoring "healthy" or "faulty".

Slide 28 — Drop the denominator: it is the same for every class

$$\hat{y} = \arg\max_c \, P(x \mid y = c)\, P(y = c)$$

To *choose* a class we only need to know which score is larger, and dividing both scores by the same positive number does not change which one is larger. So the classifier compares the numerators and picks the larger.

A natural objection is that the scores are then no longer probabilities. That is true, but nothing is lost: the denominator is the sum of the numerators over all the classes, so dividing each score by that sum gives the probabilities back exactly. Slide 36 does this with numbers. Where Naive Bayes does lose the meaning of its probabilities is inside the numerator, through the assumption on the next slide.

Worked example — one pump, two classes, with and without the denominator

Priors from the 1,200 pumps: $P(\text{faulty}) = 735/1200 = 0.6125$, $P(\text{healthy}) = 0.3875$. Suppose the readings x of one pump have likelihood 0.02 under faulty and 0.05 under healthy.

class	prior × likelihood	÷ $P(x)$ = posterior
faulty	$0.6125 \times 0.02 = 0.01225$	0.387
healthy	$0.3875 \times 0.05 = 0.01938$	**0.613**

$P(x) = 0.01225 + 0.01938 = 0.03163$, the same divisor for both rows. Dividing changes the numbers into probabilities but cannot change which is larger: the decision is already made in the middle column. Here the rarer class wins, because its likelihood is two and a half times larger.

Slide 29 — The assumption: independent, given the class

Given the class, the features are independent of one another. The two words "given the class" carry the whole meaning, and they are the subject of the questions below.

What the assumption buys: one n-dimensional density becomes n one-dimensional densities, each estimated from the rows of one class. A one-dimensional density needs little data, and training

becomes a single pass over the data. In scikit-learn the model is `GaussianNB()`; after `fit()`, the means it estimated are in `theta_` and the variances in `var_`.

Question — what does "independent given the class" mean, and why did we assume it for the pumps?

> *Relating to Naive Bayes, explain again why "given the class, the features are independent of each other": what does being independent mean, and why did we assume it in the first example?*

Independent means that knowing one variable tells you nothing about the other. For two features x_1 and x_2, independence means $P(x_2 \mid x_1) = P(x_2)$: learning the value of x_1 does not change what you expect for x_2. Equivalently, the joint probability is the product of the two separate ones, $P(x_1, x_2) = P(x_1)\,P(x_2)$.

Independent given the class is a weaker and different statement. It says that the equality holds *inside each class*:

$$P(x_1, x_2 \mid y = c) = P(x_1 \mid y = c)\,P(x_2 \mid y = c).$$

Inside the healthy pumps, knowing the vibration tells you nothing more about the pressure; inside the faulty pumps, the same. But across the whole dataset the two features may well be related, because both depend on the class. Knowing that x_1 is large may suggest the pump is faulty, and that in turn changes what you expect of x_2. Naive Bayes allows this kind of dependence, the dependence that runs *through* the class. It forbids any other kind.

Why we assumed it for the pumps: we did not assume it because we believed it. We assumed it because it is what *defines* Naive Bayes, and it is what turns an impossible estimation problem into an easy one. Whether it is true for the pumps is a separate question, and the lesson measures it on slide 40. The answer is that it is false, and the model works anyway; slide 41 explains why.

Question — an example of features that really are independent given the class

> *Can you give me an example of features that really are independent given the class y?*

The cleanest way to be sure is to build the data so that it is true by construction. Take electronic components, each either good or defective, and two measurements:

- x_1: whether the component runs hot when tested;
- x_2: whether its packaging is scratched.

Suppose the scratches come only from shipping, a cause that has nothing to do with the temperature. Then, among the defective components, knowing that one runs hot does not change the chance that its packaging is scratched. If 0.8 of defective components run hot and 0.3 of them have scratched packaging, the fraction that does both is $0.8 \times 0.3 = 0.24$, exactly the product. The same must hold separately among the good components.

A purely numerical version: generate class 0 with x_1 drawn from a bell centred at 0 and x_2 drawn, separately, from a bell centred at 5; generate class 1 with the two bells centred at 3 and 10. Within each class the two draws are made independently, so the assumption holds exactly.

The instructive part is what the *whole* dataset looks like. Plot all the points together and x_1 and x_2 appear correlated: where x_1 is large you are probably looking at class 1, and in class 1 x_2 is large too. Split the plot by class and each cloud is round, with no relation inside it. **Once you know**

the class, knowing x_1 no longer helps you predict x_2. That is exactly what "independent given the class" means, and why it is a different statement from "independent".

Slide 30 — The assumption, written: a product of n densities

$$P(x \mid y = c) = \prod_{j=1}^{n} P(x_j \mid y = c)$$

On the left, one n-dimensional density; on the right, n one-dimensional ones. The difference is a counting argument. To estimate the left side directly you need enough examples to fill an n-dimensional space, which is the demand the curse of dimensionality makes impossible. To estimate the right side you need enough examples to draw n separate histograms, which is easy. That is the bargain: an assumption that is usually false, exchanged for an estimation problem that can actually be solved.

Worked example — what the assumption saves, counted

Take n yes/no readings per pump. Describing $P(x \mid y = c)$ exactly needs a probability for every combination of answers; under the naive assumption it needs one probability per reading:

readings n	joint distribution, per class	naive, per class
2	$2^2 - 1 = 3$	2
10	$2^{10} - 1 = 1,023$	10
30	$2^{30} - 1 = 1,073,741,823$	30

With 30 readings the joint table has over a billion cells per class, almost all of them never seen in any training set. The naive model needs 30 numbers, each estimated from every row of the class. That is the bargain: it is false, and it is learnable.

Slide 31 — Gaussian: each factor is a bell curve (optional)

$$P(x_j \mid y = c) = \frac{1}{\sqrt{2\pi\sigma_{jc}^2}} \exp\left[-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2}\right]$$

The factorisation leaves one choice open: the shape of each one-dimensional density. **Gaussian** Naive Bayes answers "a bell curve", with its own centre μ_{jc} and its own width σ_{jc} for every feature j in every class c.

So training is not an optimisation at all. It computes the class priors, and a mean and a variance per feature per class, in one pass over the data: no gradient descent, no learning rate, no iterations. For the pumps, with two features and two classes, that is two priors, four means and four variances: ten numbers.

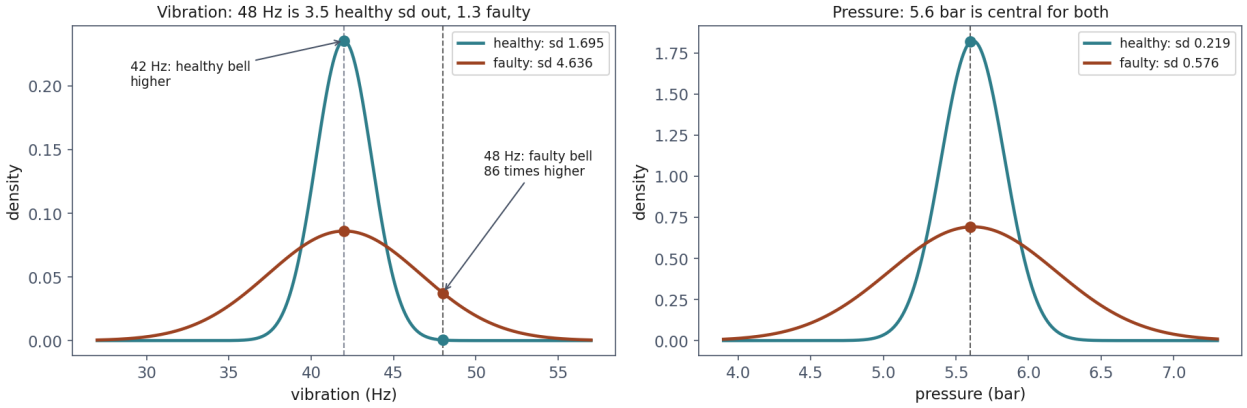
Slide 32 — Training computes ten numbers: the widths separate the classes

Class	Prior	Vibration mean	Vibration sd	Pressure mean	Pressure sd
Healthy	0.3875	41.986 Hz	**1.695**	5.617 bar	**0.219**
Faulty	0.6125	42.015 Hz	**4.636**	5.616 bar	**0.576**

Read the means first: the two classes have **the same centre**, within a few hundredths of a hertz. What differs is the width. Healthy pumps form a narrow bell around the design point, faulty pumps a wide one: the disc and the ring of slide 5, seen one feature at a time.

This also explains why logistic regression failed. A linear model separates classes by placing a boundary between their means, and here there is nothing between the means. Naive Bayes compares widths instead, and widths are where the information is.

Slide 33 — A pump at 48 Hz, 5.6 bar: there the faulty bell is 86 times higher



Follow the dashed line at 48 Hz up the left panel: the narrow healthy bell has fallen almost to nothing, the wide faulty one is still well up — 86 times higher. At 42 Hz the narrow bell is on top. On the right, 5.6 bar is central for both classes and barely discriminates; the narrow healthy bell peaks near 1.82, above 1.

Take a pump at 48 Hz and 5.6 bar. 48 Hz is 6 Hz from the design point. For the narrow healthy bell, whose standard deviation is 1.695 Hz, that is three and a half standard deviations, far into the tail. For the wide faulty bell, standard deviation 4.636 Hz, it is barely more than one. So at 48 Hz the faulty bell is 86 times higher than the healthy one, and this single term decides the pump: $P(\text{faulty}) = 0.981$. At the design point itself, 42 Hz, the order reverses and the narrow bell is on top; that pump comes out healthy, with $P(\text{healthy}) = 0.820$.

On the pressure panel, 5.6 bar sits at the centre of both bells, so this reading barely discriminates. Note the height of the narrow healthy bell there: about 1.82, **greater than 1**. That is not an error. A density is not a probability: it is a probability *per unit* of the measurement, and a bell only 0.219 bar wide must rise high so that its total area is still 1.

Slide 34 — Take logs: the product becomes a sum (optional)

$$\ln\left[P(y=c)\prod_j P(x_j \mid y=c)\right] = \ln P(y=c) + \sum_j \ln P(x_j \mid y=c)$$

Why take logarithms at all? With hundreds of features the product is hundreds of small numbers multiplied together, and a computer rounds it to exactly zero (this is called **underflow**). Already 0.5^{300} is about 5×10^{-91}, and real likelihoods are far smaller than 0.5. Every class then scores zero and nothing can be compared.

Two facts repair this. The logarithm is an increasing function, so the class with the largest product is also the class with the largest logarithm, and the decision does not change. And the logarithm of a product is the sum of the logarithms, so a product that underflows becomes a sum of ordinary negative numbers. This is the same reason lesson 4 worked with the log-likelihood.

Slide 35 — 48 Hz, 5.6 bar: scores and their logs (optional)

Class		Prior	Vibration density	Pressure density	Product, or sum of logs
Healthy	value	0.3875	0.000433	1.818	0.000305
Healthy	ln	−0.948	−7.745	+0.598	−8.095
Faulty	value	0.6125	0.0374	0.692	0.01585
Faulty	ln	−0.490	−3.286	−0.368	**−4.144**

This table is the whole of prediction. Read each class as two rows: the "value" row multiplies the prior by the height of that class's bell at each reading, and the "ln" row adds the same numbers after taking logarithms. You can check one with a calculator: $0.3875 \times 0.000433 \times 1.818 = 0.000305$, and $e^{-8.095} = 0.000305$ as well.

Faulty has the larger total, −4.144 against −8.095, so faulty is the prediction. Note what was *not* used: no training row was consulted, only the ten stored numbers. That is the exact opposite of k-NN, for which prediction is the whole cost.

Slide 36 — From the two totals to probabilities (optional)

Step	Healthy	Faulty
1. Log total, previous slide	−8.095	−4.144
2. Undo the log: e^{total} = prior × densities	0.000305	0.01585
3. Divide by their sum: 0.01616 = $P(x)$	**0.019**	**0.981**

Step 2 gives back the two numerators of Bayes' rule. Step 3 needs one fact: why is their sum equal to $P(x)$? Because a pump's readings come either from a healthy pump or from a faulty one, so, by

the law of total probability,

$$P(x) = P(\text{healthy})\,P(x \mid \text{healthy}) + P(\text{faulty})\,P(x \mid \text{faulty}),$$

which is exactly the two scores added. Dividing each score by it gives probabilities that add to one: $0.000305/0.01616 = 0.019$ and $0.01585/0.01616 = 0.981$. This is the denominator that slide 28 dropped, recovered.

Slide 37 — Software uses only the gap between the logs (optional)

The same result reached without computing the two scores, which is how software does it. Divide the numerator and the denominator of $0.01585/(0.01585 + 0.000305)$ by 0.01585, and the fraction becomes $1/(1 + 0.000305/0.01585)$. The two scores are e raised to their totals, and a ratio of exponentials is the exponential of the difference, $e^a/e^b = e^{a-b}$. So the ratio is $e^{-8.095+4.144} = e^{-3.951} = 0.0192$, and $P(\text{faulty}) = 1/1.0192 = 0.981$.

Only the **gap** between the two totals matters, never the totals themselves, and that is the practical point. With hundreds of features the totals reach values like -800, e^{-800} is exactly zero in floating point, and the direct fraction becomes $0/0$. The gap stays an ordinary number. Read it as odds: $e^{3.951}$ is about 52, so faulty is 52 times as likely as healthy, which splits the probability as $52/53 = 0.981$ and $1/53 = 0.019$. The formula $1/(1 + e^{-\Delta})$, with Δ the gap, is lesson 4's sigmoid.

Slide 38 — A cheap model, built on an assumption that is almost never true

The slide holds two halves, and both matter. The bargain is real: one pass of training, and almost no extra cost per added feature, which is why Naive Bayes can handle ten thousand features on a few thousand rows.

The assumption is false almost everywhere. Vibration and pressure are both driven by the operating point of the same machine. In text, the words "New" and "York" appear together far more often than independence would allow. In medicine, the symptoms of one disease come in clusters. So the useful question is not "does the assumption hold?" but "**when does being wrong about it cost nothing?**" The key is that a *false assumption* and a *wrong classifier* are different things: to classify, the model only needs the right class to come out on top, not the probabilities to be correct.

Slide 39 — Naive Bayes on the pumps: 0.933

Model	Accuracy
Majority baseline	0.613
Logistic regression	0.613
Gaussian Naive Bayes	**0.933**
k-NN, k = 5	0.944

One point behind k-NN, and a very long way ahead of the linear model, from a method that trains in a single pass and rests on an assumption we have just called almost never true. The right reaction

is not to stop and celebrate but to ask *why* it worked. A model that works because its assumption holds will keep working on similar data; a model that works *despite* its assumption failing may not, and you want to know which case you are in.

Question — can a good accuracy tell me that the assumption was true?

> *So in theory I could always take the Naive Bayes assumption as true, compute an accuracy, and from the result deduce whether the assumption was really satisfied, correct?*

The first half is right, the second is not. You can use Naive Bayes without knowing whether its assumption holds; that is normal practice. But **the accuracy cannot tell you whether the assumption holds**, and the pumps are built precisely to show it: accuracy 0.933, and an assumption that slide 40 proves false.

The reason is in what classification needs. The model predicts $\arg\max_c P(y = c \mid x)$, the class with the largest score. Its estimated densities can be wrong, even badly wrong, while the class with the largest score is still the correct one. On the pumps that is exactly what happens (slide 41).

So there are two separate questions, answered by two separate tools:

- **Does the model predict well?** Cross-validation answers this.
- **Is the independence assumption true?** Only a study of how the features relate *within each class* answers this. The accuracy does not.

A low accuracy does not prove the assumption false either: the model could fail for other reasons, for instance features that carry no information.

Slide 40 — Uncorrelated within each class, but not independent

Correlation, within the class	vibration and pressure	their squared distances from the design point
healthy pumps	−0.006	+0.168
faulty pumps	−0.049	**−0.420**

The middle column suggests the assumption holds: within each class the two readings are uncorrelated. But **correlation only detects straight-line dependence**. The faulty pumps fill a ring around the design point. On a ring, a pump whose vibration is far out must have a pressure close to the centre, otherwise it would lie outside the ring. That is a dependence, but a symmetric one: far out to the left and far out to the right both go with a central pressure, so a straight-line summary averages it to zero.

The third column makes it visible. Square each reading's distance from the design point, so that "far out on either side" becomes "large", and among the faulty pumps the two squared distances correlate at −0.420: when one reading is far out, the other is pulled in. Uncorrelated, yes; independent, no. The assumption is false on these pumps.

Why the square and not, say, the cube? Because the dependence is about *how far* from the centre, not *which side*, so the transformation must treat −2 and +2 alike: it must be an **even** function.

The square and the absolute value are even (the absolute value gives −0.454); the cube keeps the sign, so the two sides still cancel, and it finds only −0.022.

Question — what does the correlation code do?

Can you explain this code?

```
print("correlation between the two readings")
print(f"  overall            {X.corr().iloc[0, 1]:+.3f}")
for label, name in ((0, "healthy"), (1, "faulty")):
    print(f"  within {name:<9}  {X[y == label].corr().iloc[0, 1]:+.3f}")
```

Piece by piece:

- `X` is a pandas DataFrame with two columns, vibration and pressure. `X.corr()` returns the 2 × 2 matrix of correlations between the columns. The diagonal is always 1 (each column with itself); the two off-diagonal entries are the same number, the correlation between vibration and pressure.
- `.iloc[0, 1]` picks the entry in row 0, column 1 by position: that off-diagonal number.
- `:+.3f` formats it with three decimals and an explicit sign, so `+0.168` and `-0.420` line up.
- The loop runs twice, once with `label = 0, name = "healthy"` and once with `label = 1, name = "faulty"`.
- `y == label` is a column of True and False, one per pump, and `X[y == label]` keeps only the rows where it is True: the pumps of that class. So `X[y == label].corr()` is the correlation computed **within one class**, which is what the assumption is about.
- `name:<9` pads the name to nine characters, left-aligned, so the two lines align.

The output is −0.046 overall, −0.006 within healthy and −0.049 within faulty. All three are close to zero, and that is the trap the code sets on purpose: the natural conclusion, "the assumption holds", is wrong, because the code checked only straight-line dependence.

Question — what do the squared-distance results say, and where do 0.784 and 1.309 come from?

> *What do these results say?* — followed by the output of the squared-distance cell: −0.420 within the faulty pumps, standard deviations 0.784 and 1.309, then 0.37 and 0.62 bar.
>
> *Why is the standard deviation of the pressure 0.784 when the vibration is extreme, and 1.309 when it is not?*

The cell does two things. First, the correlation of the squared distances, discussed above. Second, a more direct test of dependence: take only the faulty pumps, split them into those whose vibration is **extreme** (the fifth furthest from the centre, which means more than 6.2 Hz away; 147 pumps) and the others (588), and measure how spread out the pressure is in each group.

Both 0.784 and 1.309 are measured, not derived from a formula. Their difference has a geometric reason. A faulty pump with extreme vibration sits at the far left or far right of the ring, and there the ring is narrow in the pressure direction: the pressure is held close to the centre, so its spread is small. A faulty pump whose vibration is not extreme can sit at the top or bottom of the ring, where the pressure is far out in either direction, so its spread is large. The *average* pressure is the same in both groups, which is why the correlation saw nothing; the *spread* differs, and a distribution that changes when you learn the other reading is a dependent one.

The units need care. The notebook standardises each reading with the mean and standard deviation of **all** 1,200 pumps (`z = (X - X.mean()) / X.std()`), so 0.784 and 1.309 are in units of the overall standard deviation of pressure, which is 0.471 bar. Hence $0.784 \times 0.471 = 0.37$ bar and $1.309 \times 0.471 = 0.62$ bar, the two numbers the notebook prints. Multiplying by the faulty pumps' own standard deviation, 0.576 bar, would give different numbers, and would be wrong for this cell.

Question — how would you check the assumption with many features?

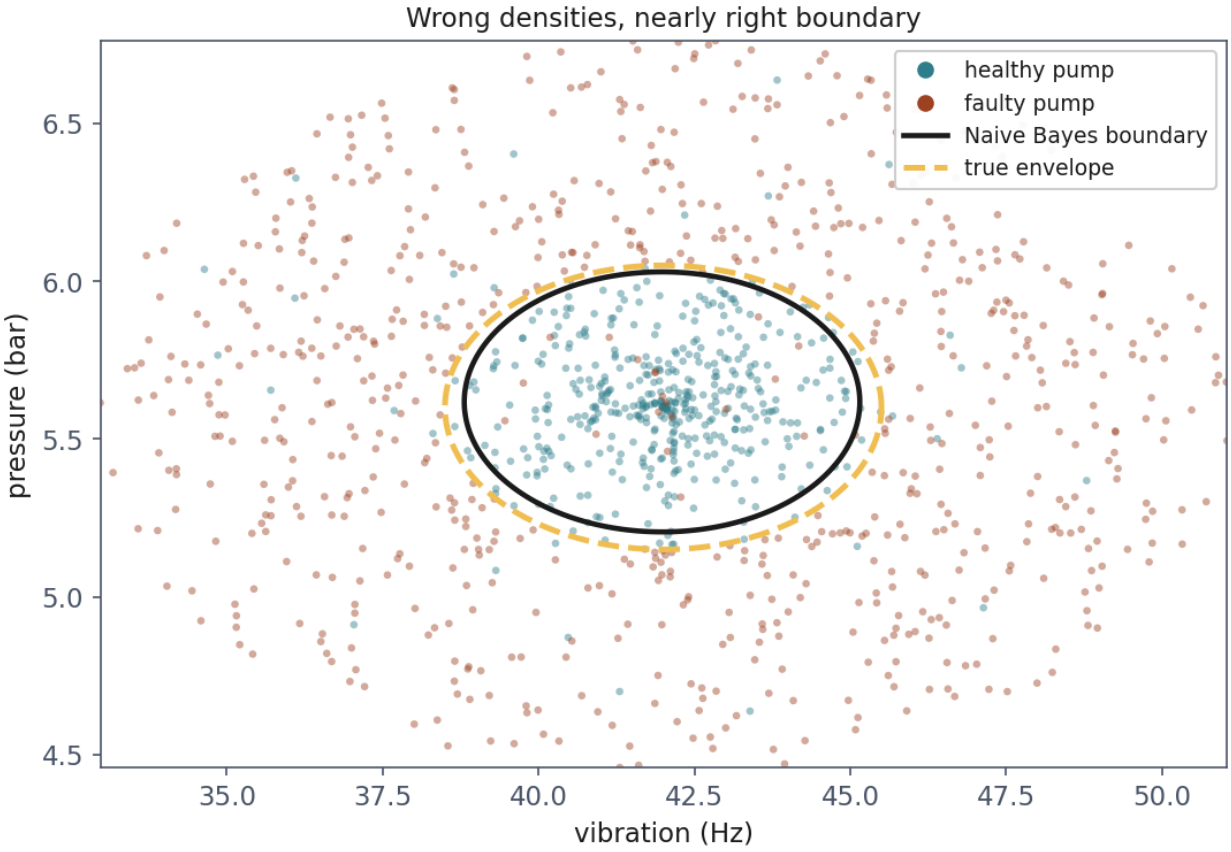
> *In the handout's examples there were only two features, so the dependence was visible in the figures too. With many more features, checking it is harder. How could it be done?*

With two features you can look at a scatter plot; with a hundred you cannot. Three levels of checking, from cheap to hard:

1. **Within-class correlation matrices.** For each class separately, compute `X[y == c].corr()`, an $n \times n$ matrix, and draw it as a heatmap. A large entry, say 0.9 between two features within one class, is strong evidence that those two are not independent given the class. But the reverse does not hold, as the pumps just showed: a correlation near zero does not prove independence.
2. **Measures that see curved dependence.** The pumps' dependence was found by squaring the readings first. A general tool, beyond this course, is **mutual information**, which is zero exactly when two variables are independent and positive for any kind of dependence. A simple case where correlation fails and mutual information does not: $x_2 = x_1^2$ with x_1 symmetric around zero has correlation zero, yet x_2 is completely determined by x_1.
3. **The hard limit.** Even if every *pair* of features is independent within a class, the full factorisation may still fail: three features can be pairwise independent and yet related as a group. Checking every pair is therefore still not a proof.

The practical conclusion: do not try to *certify* that the assumption holds. Look for evidence that it is badly broken, and then ask whether that breakage damages what you need from the model. On the pumps it is broken and costs almost nothing; on the interacting sensors of slide 42 it is broken and ruins the model.

Slide 41 — The densities are wrong, but the boundary they draw is about right



The solid ellipse is where the fitted model changes its mind, the dashed one the envelope the pumps were generated from: same shape and orientation, the model's a little smaller. The densities are wrong; the boundary they imply is nearly right.

So why 0.933? Both classes are centred on the same point, and Naive Bayes compares a narrow bell with a wide bell on each axis. Where the two scores are equal is an ellipse whose axes run along the vibration and pressure directions, and the true envelope that generated the data is also such an ellipse. The model's ellipse has edges at about 3.17 Hz and 0.41 bar from the centre, against a true 3.5 Hz and 0.45 bar: the same shape, about a tenth smaller. **The densities are wrong; the boundary they imply is nearly right.** A false assumption is cheap when it leaves the boundary the right shape.

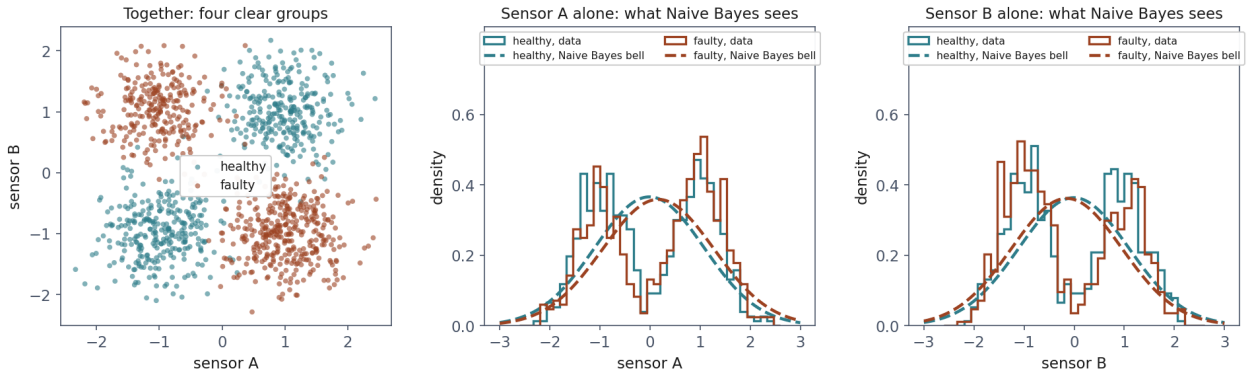
Slide 42 — A second dataset: the signal is an interaction

A second pair of sensors on the same fleet, with a rule that is perfectly reasonable engineering: both readings high is the designed high-load mode, both low is idle, and **exactly one high** is a mismatch between demand and delivery, which is the fault.

Ask what each sensor says *on its own* under this rule. The answer is nothing: every value of sensor A appears in both classes about equally often, because whether a high A means "faulty" depends entirely on B. That is what an **interaction** is: the information lives in the combination of the

features, not in any one of them. And it is exactly the structure that the independence assumption cannot represent.

Slide 43 — Each sensor alone: the two classes look the same



Left: together, the two sensors show four clear groups, each class owning two opposite quadrants. Middle and right: what Naive Bayes gets to see — each sensor alone. Solid outlines are the data, two humps each and the same two humps for both classes; dashed curves are the single bell per class that Naive Bayes fits, the two almost on top of each other.

On the left, the two sensors together: four clean groups, each class owning two opposite quadrants. Any method that can draw a curved boundary will find the rule. In the middle and on the right, each sensor alone, which is all Naive Bayes ever looks at. Each sensor is either low or high, so each outline has two humps, and healthy and faulty have the *same* two humps. The dashed curves are the bells the model fits: one per sensor per class. A single bell cannot follow two humps, so it sits in the valley between them, and the two classes' bells come out almost on top of each other, centre near 0 and standard deviation about 1.1 for both. On the pumps the widths carried the class; here nothing does. And more data does not help: with a million rows the bells still coincide, because the model cannot represent what is being asked of it.

Question — what do the interacting sensors' numbers show?

> *1200 pumps, 628 faulty (52.3%). How much each sensor tells you on its own: correlation of sensor_a with the label +0.081, of sensor_b −0.064. And the means per class: healthy −0.012 and −0.006, faulty 0.167 and −0.149.*

They show, in numbers, what the middle and right panels show in pictures: **each sensor on its own carries almost no information about the class.**

- 628 faulty pumps out of 1,200 is 52.3%, so always answering "faulty" scores 0.523. That is the baseline for this dataset.
- The correlations of each sensor with the label, +0.081 and −0.064, are close to zero: there is almost no straight-line relation between one sensor's value and being faulty.
- The class means differ by 0.18 on sensor A and 0.14 on sensor B, against a spread of about 1.1 within each class. That gap is small compared with the spread, and it is a product of this particular finite sample, not of the rule that generated the data.

The notebook's next cell shows where the information really is. Count the pumps in each quadrant: A high and B high, 273 healthy and 9 faulty; A high and B low, 10 healthy and 359 faulty; A low and B high, 11 healthy and 253 faulty; A low and B low, 278 healthy and 7 faulty. Together, the two sensors classify almost perfectly. Apart, they say almost nothing. And within each class the two sensors are strongly tied, +0.821 among the healthy pumps and −0.826 among the faulty: the assumption fails as badly as it can.

Worked example — the interaction, counted

100 pumps, 25 in each cell. A pump is faulty when **exactly one** reading is high:

	pressure low	pressure high
vibration low	25 healthy	25 faulty
vibration high	25 faulty	25 healthy

What Naive Bayes estimates is one column at a time, within each class:

- $P(\text{vibration high} \mid \text{faulty}) = 25/50 = 0.5$, and $\mid$ healthy also 0.5;
- $P(\text{pressure high} \mid \text{faulty}) = 0.5$, and $\mid$ healthy also 0.5.

Every factor is the same for both classes, so every pump gets the same score and the model can only fall back on the prior. The rule lives entirely in the combination, and the combination is exactly what the assumption throws away.

Slide 44 — 0.404: below the baseline, and below chance

Model	Accuracy
Majority baseline	0.523
Gaussian Naive Bayes	**0.404**
Logistic regression	0.393
SVM, linear kernel	0.606
k-NN, k = 5	0.967
SVM, RBF kernel	0.972

First, the problem is not hard: k-NN and the SVM with a radial basis function (RBF) kernel reach 0.967 and 0.972. It is hard only for a model that looks at one feature at a time.

Second, 0.404 is **below chance**: below the majority baseline of 0.523, and below a coin flip. How can a model with no information do worse than guessing? Because it does not know it has no information. The small gaps between the class means, 0.18 and 0.14, are the only per-feature evidence it has, so it follows them. They are accidents of this sample, and here they point the wrong way. **A model with no signal does not sit politely at 50%: it follows whatever accidental structure it can find.**

Slide 45 — Its probabilities cannot be trusted

On the interacting sensors, Naive Bayes is on average 0.567 sure of itself when it is right and 0.555 when it is wrong: practically the same. Its reported probability carries no information about whether to trust a given prediction.

The more common complaint about Naive Bayes runs the other way. When several features carry the *same* evidence, near-copies of one another such as ten words that always appear together in a document, multiplying their probabilities counts that evidence ten times. The posterior is pushed to the extreme, and the model reports 0.999 on problems it gets wrong a fifth of the time. Either way, the lesson is the one from lesson 5: **ranking and calibration are different properties**. A model can put examples in a useful order and still attach numbers to them that mean nothing as probabilities.

Question — is this the same distinction as the area under the curve against calibration?

> *Referring to slide 45, when you say "ranking and calibration are different properties", is it the same point we made with the area under the curve (AUC) for logistic regression? AUC can be high while the model produces uncalibrated probabilities, correct?*

Yes, it is exactly the same distinction. Two separate questions can be asked of a model's probabilities:

- **Ranking** (also called discrimination): does the model place the positive examples above the negative ones? The area under the receiver operating characteristic (ROC) curve, AUC, measures this, and only this.
- **Calibration**: when the model says 0.8, are about 80% of those cases really positive?

Two models can have the same ranking and very different calibration. Take four examples, two negative and two positive. Model A gives them 0.10, 0.20, 0.70 and 0.90; model B gives 0.40, 0.45, 0.55 and 0.60. Both put the four in the same order, with both negatives below both positives, so both have an AUC of 1. But model B's numbers are timid: it is right every time and never says more than 0.60. AUC cannot tell the two apart, because it looks only at the order.

The practical consequence is the one on the slide's notes. If all you need is an order (which documents to read first, which pumps to inspect first), the ranking is what matters. If a decision depends on the probability itself, as in lesson 4's cost-based threshold, the numbers must be calibrated, and Naive Bayes's are not to be trusted.

Question — what does section 5 of notebook 2 mean?

> *What does section 5 of notebook 2 mean? "mean confidence when right: 0.567, mean confidence when wrong: 0.555, highest confidence reached: 0.673. The two numbers are nearly identical…"*

First, what "confidence" is here. For each test pump the model gives $P(\text{faulty})$; it predicts faulty when that is at least 0.5. The confidence is the probability of the class it chose, $\max(p, 1-p)$, which is always between 0.5 and 1. A prediction at 0.51 is barely sure; one at 0.99 is very sure.

Then the three lines:

- **0.567 when right, 0.555 when wrong.** A useful model should be clearly more sure when it is right than when it is wrong, so that a high confidence means "trust me". Here the two

averages differ by about a hundredth. Confidence tells you nothing about correctness.
- **0.673 at most.** Over the whole test set the model never goes beyond 0.673. It is never sure of anything, which at least is honest: on these sensors it really has no per-feature evidence.

So the probability column of `predict_proba` cannot be used to decide which predictions to trust. The notebook then contrasts this with the opposite and more common failure (the overconfident one described above), and closes with the same conclusion: the ranking can be useful while the probabilities are not.

Slide 46 — Notebook 2, live

The slide's table maps each section of the notebook to its slides and to the scikit-learn calls it uses. The notebook walks the whole story: Naive Bayes reaches 0.933 on the pumps, the within-class correlations look like zero, the squared distances show the dependence anyway, the fitted ellipse explains why the cost is small, and then the interacting sensors bring it to 0.404. The diagnostic to carry away is in its correlation cells: **measure the assumption, do not assume it.** An experiment worth trying if time allows: give Naive Bayes the product of the two sensors as a third column. The product is positive when both are high or both are low, negative when exactly one is high, so it carries the whole interaction in a single feature, and the model recovers. That is the choice between engineering a feature and choosing a different model, in one cell.

Question — what do the notebook's conclusions mean?

> *Comment on the notebook's conclusions: "Very high dimensions with little data" (text classification), "As a baseline", "When the class ranking is what you need", and the one case to avoid it, "when the signal is an interaction".*

They answer a practical question: if the assumption is almost always false, when is Naive Bayes still the right choice?

- **Very high dimensions with little data.** Classifying documents by topic, each described by the counts of fifty thousand words, with three thousand training documents. Estimating the joint density of fifty thousand counts is hopeless. Naive Bayes estimates instead one simple number per word per topic, such as how often "goal" appears in sport articles. The assumption is clearly false (after "New", "York" is far more likely), but a model that is roughly right and *can* be estimated beats a more faithful one that the data cannot support.
- **As a baseline.** Training is a single pass with no tuning, so it costs nothing to run first. If a model that took hours of feature engineering and tuning does not beat it, you have learned something important and cheaply. It does not make Naive Bayes the best model; it makes it a serious reference point, one step above the majority baseline.
- **When the ranking is what you need.** Its probabilities may be badly calibrated while their order is still useful: if a pump scored 0.99 is more suspicious than one scored 0.80, an inspection list sorted by score works even though "0.99" is not a real 99%. When a decision rule reads the probability itself ("intervene above 0.8"), this no longer holds.
- **When to avoid it: the signal is an interaction.** As on slides 42 to 44, when the class depends on a *combination* of features, a model that looks at each feature separately cannot represent the rule, and no amount of data changes that.

Part V — Support vector machines

Slide 47 — The margin: choosing among lines that all fit

When the two classes can be separated by a line, there are infinitely many lines that do it, and every one of them has zero training error. So the training error cannot choose among them; some other criterion must. Logistic regression chooses with the log loss. The support vector machine chooses differently: it takes the line with the **widest empty slab** around it, pushed out until it touches the nearest point of each class.

The reason is about generalisation, not fit. A boundary that passes close to a training point is one small change in the data away from misclassifying it. The boundary with the most room around it tolerates the most movement in the data before it changes its mind.

Question — why does logistic regression with gradient descent not find the widest margin?

> *The SVM chooses the boundary that maximises the margin; that is clear. Why does gradient descent applied to a logistic regression not manage it? Logistic regression plus gradient descent stops at some point, and the line found is not the one with the optimal margin, correct?*

The difference does not lie in gradient descent. Gradient descent is an **optimiser**: it answers "how do I move the parameters so that the cost goes down?". What differs is the **objective**, the answer to "what counts as a good solution?". Logistic regression asks for a small log loss; the SVM asks for a wide margin. Each optimiser finds what it was asked for, and the two requests have different answers.

The difference shows in what happens to a point that is already correctly classified. Write $y_i f(x_i)$ for how far point i sits on its correct side, with labels ± 1 and $f(x) = w^\top x + b$. The log loss $\ln(1 + e^{-y_i f(x_i)})$ is 0.127 at a margin of 2 and 0.049 at 3: it keeps falling, so even a point that is safely correct keeps pulling on the boundary, and every point influences the answer. The SVM's hinge loss, $\max(0,\ 1 - y_i f(x_i))$, is exactly zero once $y_i f(x_i) \geq 1$: past that point, pushing it further away is worth nothing. And the term $\frac{1}{2}\|w\|^2$ in the SVM's objective (slide 51) is what explicitly asks for the widest slab.

So "gradient descent stops before reaching the widest margin" is not the right picture. Run it as long as you like: logistic regression will converge to the line that minimises the log loss, which is in general a different line, because nothing in its objective mentions the margin. (For the curious: on perfectly separable data and with no penalty at all, the *direction* of logistic regression's coefficients does drift towards the widest-margin line as training runs forever. That is a theoretical curiosity, not something any real fit relies on.)

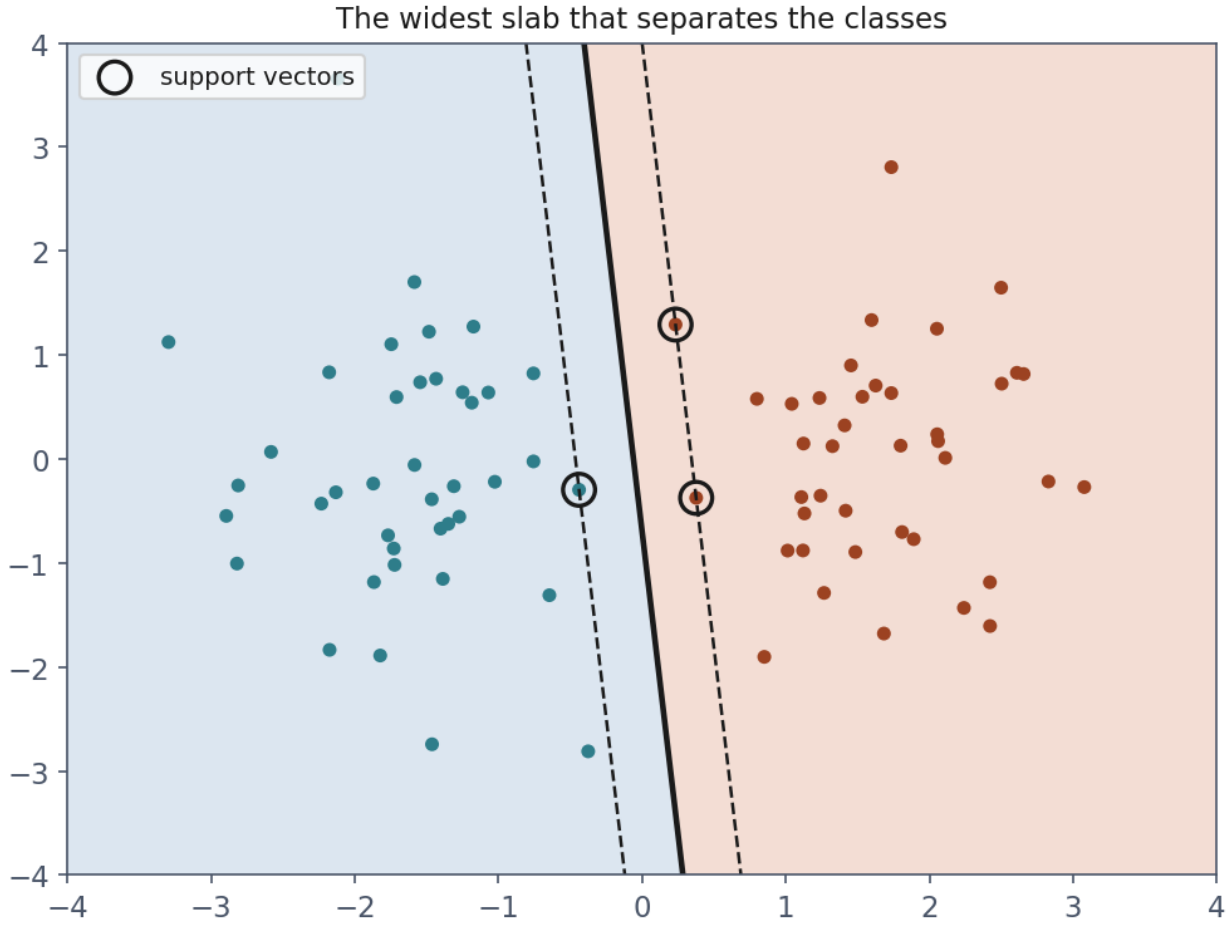
Worked example — distance to a boundary, and the width of the slab

Boundary $3x_1 + 4x_2 - 10 = 0$, so $w = (3, 4)$ and $\|w\| = 5$. A pump at $(2, 3)$:

$$\text{distance} = \frac{|3 \cdot 2 + 4 \cdot 3 - 10|}{5} = \frac{8}{5} = 1.6$$

If the closest pumps sit where $|w^\top x + b| = 1$, they are $1/5 = 0.2$ from the boundary and the slab is $2/\|w\| = 0.4$ wide. A boundary whose closest pumps were twice as far off, at 0.4, would satisfy the same rule only with $\|w\| = 1/0.4 = 2.5$, and its slab would be 0.8 wide. Under that rule a shorter w **is** a wider margin, which is why the SVM minimises $\|w\|$.

Slide 48 — 80 points in two groups: three of them fix the line (support vectors)



The solid line is the boundary, the dashed lines the edges of the slab, and the circled points the support vectors touching it. Of eighty points, three determine the answer; move any of the others and nothing changes.

Of eighty points, three determine the boundary. Move any of the other seventy-seven, as far as you like, as long as it stays outside the slab, and the boundary does not move. These three are

the **support vectors**, and the name comes from this: they support the boundary, the rest do not touch it. This is new in the course. Every linear model of lessons 3 and 4 used every row, each contributing to the gradient. Two consequences: the fitted model is small (you keep the support vectors, not the dataset, unlike k-NN), and an outlier near the boundary matters a great deal while one far from it matters not at all.

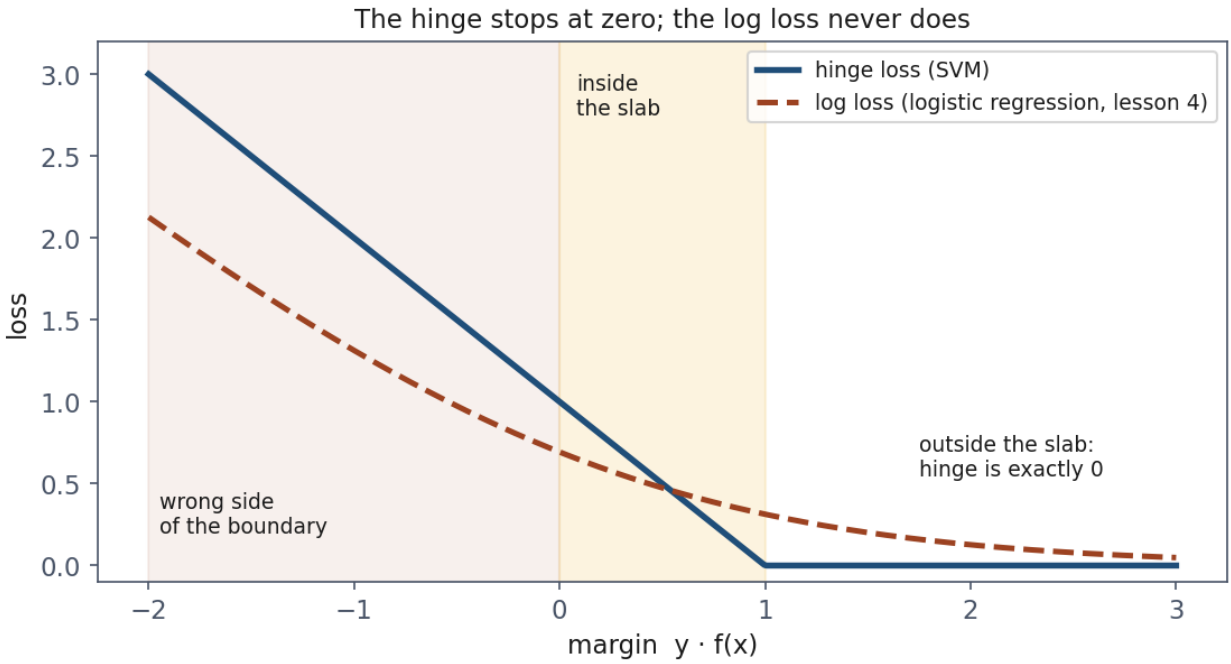
Slide 49 — Not separable? Charge for the violations

With 4% of the labels flipped, some faulty pumps sit among the healthy ones, and no slab can have all points outside it. The strict problem has no solution at all, not merely a bad one. So the SVM that is actually used everywhere is the **soft-margin** version: each point i may violate the margin by an amount $\xi_i \geq 0$ (read "xi"), and every unit of violation is charged at a price C.

- $\xi_i = 0$: the point is outside the slab, on its own side, or exactly on its edge.
- $0 < \xi_i < 1$: inside the slab, but still on the correct side of the boundary.
- $\xi_i > 1$: on the wrong side of the boundary, so misclassified.

These three cases come back twice: in the weights that training produces (slide 61) and in who gets a vote at prediction (slide 62).

Slide 50 — The hinge loss stops at zero (optional)



Loss against the margin $y \cdot f(x)$. Right of 1 — outside the slab on its own side — the hinge is exactly zero, while the log loss keeps a small charge: 0.127 at a margin of 2, 0.049 at 3. That zero is where support vectors come from.

The horizontal axis is the margin $y \cdot f(x)$: large and positive for a point safely on its own side,

negative on the wrong side. Left of zero, both losses are large. Between 0 and 1, inside the slab, the hinge still charges, falling along a straight line. Right of 1 the hinge is exactly zero, while lesson 4's log loss is still charging. **A point with zero loss stops pulling on the answer**, and that is where support vectors come from.

Worked example — five pumps, two losses

The margin of a pump is $y \cdot f(x)$: positive when it is classified correctly, larger the further it sits on its own side.

margin $yf(x)$	where it sits	hinge $\max(0, 1 - yf)$	log loss $\log(1 + e^{-yf})$
2.0	outside the slab, right side	**0.00**	0.127
1.0	on the edge	**0.00**	0.313
0.5	inside the slab	**0.50**	0.474
0.0	on the boundary	**1.00**	0.693
-1.0	wrong side	**2.00**	1.313

Past a margin of 1 the hinge is exactly zero: that pump stops influencing the boundary at all. The log loss never reaches zero, so every pump keeps a vote. That zero is where support vectors come from.

Slide 51 — The SVM objective: small w, plus C times the slack (optional)

$$\min_{w,b,\xi} \ \tfrac{1}{2}\|w\|^2 + C\sum_i \xi_i$$

subject to $y_i(w^\top x_i + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$ for every training point. The first term wants a wide slab: the width of the slab is $2/\|w\|$, so a small $\|w\|$ means a wide slab. The second term wants few and small violations. C sets the exchange rate between the two. The half in front of the norm is there only to make the derivative tidy, as in lesson 3's cost.

Question — is training the minimisation of the hinge loss, and what is ξ_i a function of?

> *In the SVM it is not clear to me how training happens. Is the hinge loss minimised?*
>
> *So $\xi_i = \max(0, 1 - y_i f(x_i))$ is always a function of y_i and x_i, correct?*

Yes, with one addition: the hinge loss is minimised **together with** the term that asks for a wide margin. The constraint says $\xi_i \geq 1 - y_i f(x_i)$ and $\xi_i \geq 0$, and since the objective wants every ξ_i as small as possible, at the optimum each one takes the smallest allowed value,

$$\xi_i = \max(0,\ 1 - y_i f(x_i)),$$

which is exactly the hinge loss of point i. Substituting it removes the ξ_i and leaves a problem in w and b alone:

$$\min_{w,b} \ \tfrac{1}{2}\|w\|^2 + C\sum_{i=1}^{m} \max(0,\ 1 - y_i(w^\top x_i + b)).$$

So training is "hinge loss plus a margin term". With the hinge loss alone, any line that put every point beyond $y_i f(x_i) = 1$ would do, and nothing would ask for the widest one.

On the second question: ξ_i depends on the training example (x_i, y_i) **and on the current boundary** w, b, because $f(x_i) = w^\top x_i + b$. During training x_i and y_i are fixed while w and b change, so ξ_i changes with them. It is not a property of the point; it is how much the point violates the margin *of the current boundary*. For a faulty pump ($y_i = +1$): if $f(x_i) = 2$, then $\xi_i = 0$; if $f(x_i) = 0.4$, then $\xi_i = 0.6$, correct but inside the slab; if $f(x_i) = -0.5$, then $\xi_i = 1.5$, misclassified.

Question — so some points are accepted inside the margin, or even on the wrong side?

> *Slide 51: what does "find a margin as wide as possible, but pay for the points that violate it" mean? So we accept that some points are inside the margin, or even on the opposite side?*

Yes, exactly. That is what "soft" means: the model is not required to classify every training point correctly, or even to keep every correct one outside the slab. Picture a single mislabelled faulty pump sitting deep among the healthy ones. To keep it outside the slab, the boundary would have to squeeze past it, leaving a very narrow slab for everyone else. The soft margin can instead accept a violation for that one point, pay $C\,\xi_i$ for it, and keep a wide, robust slab for the rest. Whether that trade is worth it is decided by C, which is the next slide.

Slide 52 — C is the price of a training error

Large C makes violations expensive, so the model works hard to classify every training point, at the cost of a narrower margin: low bias, high variance. Small C accepts some errors in exchange for a wider, calmer boundary. It is the third appearance of the same dial: λ in lesson 3, k earlier today, C now. Every model has a knob that trades fitting this data against surviving the next.

Mind the direction. A large λ means *more* regularisation; a large C means *less*. They run opposite ways, and scikit-learn's `C` in `LogisticRegression` follows the same inverted convention as the SVM's.

Question — in what sense is the margin "deformed"? Is it not always straight?

> *On slide 52 you say "I prefer to narrow and deform the margin in order to classify the training points correctly"… in what sense deform the margin? Is it not always linear?*

Right: for a linear SVM the word is wrong. The boundary $w^\top x + b = 0$ and the two edges of the slab, $w^\top x + b = +1$ and $w^\top x + b = -1$, are three parallel lines in two dimensions, and they cannot bend. What a larger C can do is accept a larger $\|w\|$, which makes the slab **narrower** (its width is $2/\|w\|$), and change w and b so that the lines **shift and rotate** to accommodate more training points. Narrower, shifted or rotated: never curved.

The image of a boundary that bends becomes accurate only with a kernel. With the radial basis function kernel the boundary is still a flat plane, but in the lifted space; drawn back in the original plane it can curve, and a large C, especially with a large γ, lets it wrap around individual training points. Slide 66 shows exactly that.

Worked example — two candidate boundaries, three prices

Boundary W is wide, $\|w\| = 1$, but two pumps violate it, with slack totalling 1.5. Boundary N is narrow, $\|w\| = 3$, and violates nothing. The SVM minimises $\frac{1}{2}\|w\|^2 + C\sum \xi_i$:

C	cost of W	cost of N	chosen
0.1	$0.5 + 0.1 \times 1.5 = 0.65$	$4.5 + 0 = 4.50$	**wide**
1	$0.5 + 1 \times 1.5 = 2.00$	$4.5 + 0 = 4.50$	**wide**
10	$0.5 + 10 \times 1.5 = 15.50$	$4.5 + 0 = 4.50$	**narrow**

Cheap violations buy the wide, calm boundary; expensive ones force the narrow boundary that bends to every training point. Large C means **less** regularisation — the direction that catches people out.

Slide 53 — The best straight line is still a straight line

Model	Accuracy	Support vectors
SVM, linear kernel	**0.613 ± 0.000**	947 of 1,200 (**79%**)
SVM, RBF kernel	0.947 ± 0.005	278 of 1,200 (23%)

The linear SVM scores the base rate, exactly like logistic regression. The margin is a better way of *choosing among straight lines*; it does not give you anything other than a straight line. When no line works, the best line is still useless: **a better criterion cannot rescue an inadequate family of models.** The same algorithm with one argument changed, `kernel="rbf"`, reaches 0.947, the best number of the lesson.

Slide 54 — Many support vectors: a free warning

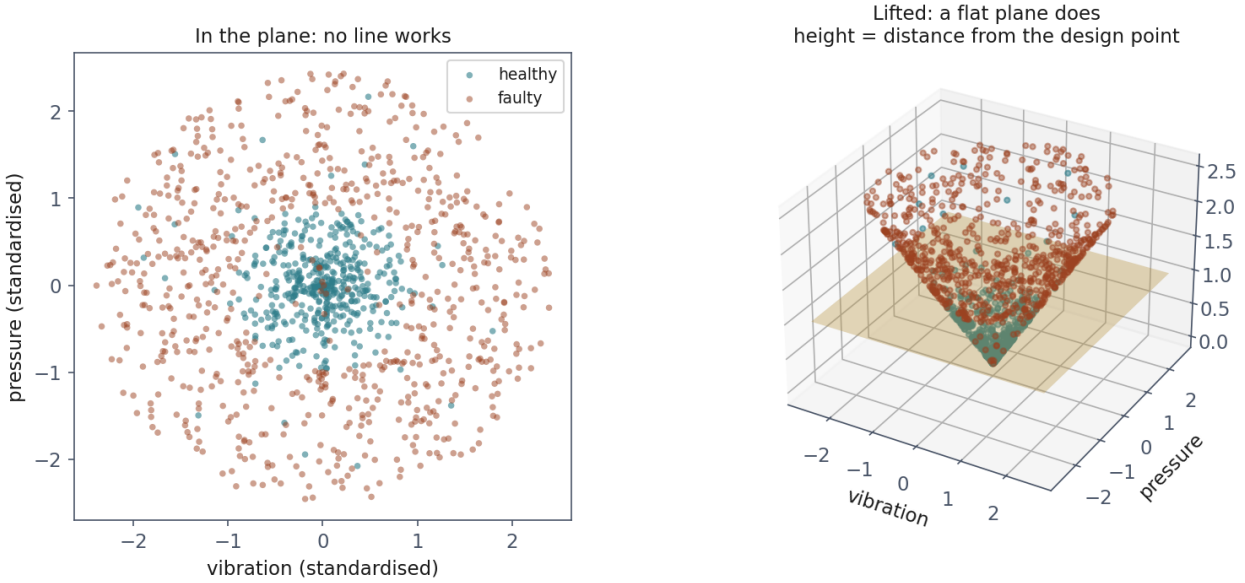
A support vector is a point on the margin or inside it. If 79% of the training set is in that position, there is no empty space between the classes at all: the model has been forced to draw a boundary through a crowd. With the RBF kernel only 23% are, because the model found real room. The diagnostic costs nothing: after fitting, compare `len(model.support_)` with the number of training rows. A high fraction usually means the kernel is wrong for the geometry, and you learn it before any cross-validation. It also predicts the cost of prediction, since each support vector is one kernel evaluation per query.

Slide 55 — A new coordinate makes the classes separable

Take the scatter plot of slide 5 and add a third coordinate: each pump's distance from the design point. Lift every point to that height. Healthy pumps rise a little, faulty ones a lot, and a flat horizontal plane now separates them. **Separability is not a property of the data alone; it is a property of the data and the coordinates you measured it in.** The unsatisfying part is

that we chose this lift because we already knew the faults were radial. In a real problem you do not know, and inventing the right coordinate is the hard part. That is the difficulty the kernel trick removes.

Slide 56 — The same pumps, lifted



The same 1,200 pumps twice: in the plane where they were measured, and lifted by their distance from the design point. The gold plane does what no line could.

Nothing was added: no new sensor. The third coordinate is computed from the two readings already there. But this lift was three-dimensional and we could guess it. The spaces that work in general are enormous, sometimes infinite-dimensional, and computing every point's coordinates there is impossible. The next slides show that it is also unnecessary.

Slide 57 — The SVM's solution: a weighted sum of pumps (optional)

$$w = \sum_{i=1}^{m} a_i \, y_i \, x_i, \qquad a_i \geq 0$$

This slide is the bridge between "lift the data" (slide 56) and "the new coordinates are never computed" (slide 58). It is the result of what the handout calls the **dual formulation**, and its message fits in one sentence: **the best w is not a new vector the optimisation invents, but a weighted sum of the training points themselves**, one weight a_i per pump. (Most texts write α_i for these weights; this course keeps α for the learning rate, so they are a_i here.)

The picture: the boundary is a tent held in place by the points pressing against it, and a_i is how hard pump i presses. A pump comfortably on its own side of the slab presses on nothing, so its weight is exactly zero. Only the pumps on or inside the slab, the support vectors, get a positive weight; that is where the 79% and 23% of slide 54 come from.

Now the consequence that matters. Put this w into the boundary:

$$w^\top x + b = \sum_{i=1}^{m} a_i \, y_i \, (x_i \cdot x) + b.$$

Every training point is still there, but only ever **multiplied against another point**: $x_i \cdot x_j$ during training, $x_i \cdot x$ at prediction. No coordinate ever appears on its own. So if we want to work in a lifted space $\phi(x)$, we never need $\phi(x)$ itself; we need only $\phi(x_i) \cdot \phi(x)$, and a function that returns that single number is enough. That function is a kernel.

Question — what is the dual formulation, intuitively?

> *The step from slide 56 to 57 is not very clear to me. It talks about the dual formulation of the SVM but does not explain what it is. Clarify it intuitively.*

Primal and **dual** are two ways of stating the same optimisation. The **primal** problem is the one on slide 51: find w and b directly, $n + 1$ unknowns, one per feature plus the intercept. The **dual** asks a different question with the same answer: find one weight a_i per *training point*, m unknowns, saying how much each point matters to the boundary.

Where the weights come from, in words. A constrained problem can be handled by attaching a price to each constraint, one per training point, paid only when that point pushes against its constraint. That price is a_i. Writing the cost with these prices and asking where its slope with respect to w is zero gives, in one line, $w = \sum_i a_i y_i x_i$: the formula on this slide. Substituting it back leaves a problem in the a_i alone, the dual, in which the training points appear only as $x_i \cdot x_j$. The handout, section 5.4, carries the derivation in full.

Why bother with a second form of the same problem? For the three reasons above:

1. it shows the boundary as a weighted vote of training points;
2. most weights are zero, so only the support vectors count;
3. the data appears only in inner products, so a kernel can replace them, and that is what makes the kernel trick possible at all.

Without the dual, slide 58's claim would be an unexplained assertion.

Slide 58 — So the new coordinates are never computed

The claim: the SVM's solution depends on the data **only through inner products between pairs of points**. So instead of computing the new coordinates, we need only a function that returns the inner product *in the new space* while computing only with the original coordinates. The radial basis function does this for an infinite-dimensional space at the cost of one exponential per pair. Why the solution depends only on inner products was shown on slide 57.

Slide 59 — The kernel trick by hand: 25 = 25 (optional)

Route	Computation	Result
Lift, then multiply	$(1,\ 2.83,\ 4)\ \cdot\ (9,\ 4.24,\ 1) = 9$ $+\ 12\ +\ 4$	**25**
Never lift	$((1,\ 2)\ \cdot\ (3,\ 1))^2 = 5^2$	**25**

The lift here is $\phi(x) = (x_1^2,\ \sqrt{2}\,x_1x_2,\ x_2^2)$, and the two points are $a = (1, 2)$ and $b = (3, 1)$. First route: lift both, $\phi(a) = (1,\ 2\sqrt{2},\ 4)$ and $\phi(b) = (9,\ 3\sqrt{2},\ 1)$, and take their inner product: $9 + 12 + 4 = 25$. Second route: never leave the plane, $a \cdot b = 3 + 2 = 5$, then square it: 25. The function $K(a, b) = (a \cdot b)^2$ *is* the inner product in the three-dimensional space, computed without going there.

And the lifted space contains x_1^2 and x_2^2, whose sum is the squared distance from the centre, the coordinate slide 55 chose by hand. A flat plane in that space is a circle or an ellipse in the original one. The kernel contains the lift we had to guess.

Question — how do I find $K(x_i, x_j)$ without knowing $\phi(x_i) \cdot \phi(x_j)$?

But how do I find a $K(x_i, x_j)$ without knowing $\phi(x_i)^\top \phi(x_j)$?

You do not go from ϕ to K; in practice you go the other way. You **choose** K from a family of functions that are known to be inner products in *some* space, and the mathematics guarantees that the space exists. You never need to write it down.

The example on this slide goes from ϕ to K only to convince you that the mechanism works: there ϕ is known and small, and the check is a calculation. The real use is the radial basis function on the next slide. For it you just compute $\exp(-\gamma\|x - x'\|^2)$ from two ordinary vectors. That this number equals an inner product in an infinite-dimensional space is a theorem (the handout, section 5.4, gives the short argument), and nobody ever constructs that space.

How do we know which functions qualify? The condition is called **Mercer's condition**, and in practical terms it says: for any set of points, the table of all values $K(x_i, x_j)$ (the kernel matrix) must be symmetric and must never produce a negative value of $\sum_i \sum_j c_i c_j K(x_i, x_j)$, whatever numbers c_i you choose. Inner products always behave this way, and any function that does is an inner product somewhere. In practice you use the standard kernels (linear, polynomial, radial basis function), which are known to satisfy it.

A caution on the picture: the radial basis function does not literally discover "the distance from the design point" that slide 55 invented. That lift was a teaching device. The radial basis function works in a much richer space, which *contains* boundaries of that shape among many others.

Slide 60 — The RBF kernel: similarity falls with distance

$$K(x, x') = \exp\left(-\gamma\|x - x'\|^2\right)$$

Read it as a **similarity**. For two identical points the exponent is zero and $K = 1$; as the points move apart, K falls towards 0. The parameter γ (gamma) sets how quickly: the similarity halves at a distance of $\sqrt{\ln 2/\gamma}$, which for the default $\gamma = 0.5$ on the standardised pumps is about 1.2

standard deviations. Small γ: wide reach, smooth boundary. Large γ: each point influences only its immediate neighbourhood.

The cost: one exponential per pair of points, so the kernel matrix has $m \times m$ entries. With 1,200 pumps that is trivial; it is why SVMs become impractical above about a hundred thousand rows, a property of the method rather than of any implementation.

Question — what does RBF stand for?

Can you remind me what RBF means?

Radial basis function. *Radial* because its value depends only on the **distance** between the two points, $\|x - x'\|$, not on the direction. Around a training point, every point at the same distance gets the same similarity, so in two dimensions the levels of equal similarity are concentric circles. It is also called the **Gaussian kernel**, because $\exp(-\gamma d^2)$ has the shape of a bell curve in the distance d.

Worked example — how similar two pumps are, by distance

$K(x, x') = e^{-\gamma\|x-x'\|^2}$ with scikit-learn's default on the standardised pumps, $\gamma = 0.5$:

distance (standard deviations)	similarity K
0.0	1.000
0.5	0.882
1.0	0.607
1.177 (half-way)	0.500
2.0	0.135
3.0	0.011

Identical pumps score 1; at 1.18 standard deviations the similarity has halved; at 3 it is about 1%. Every vote in the SVM's prediction is weighted by one of these numbers, so a pump three spreads away is, in effect, not consulted.

Slide 61 — Training: one weight per pump (optional)

Training means finding the weights a_i of slide 57. It is not gradient descent: the solver picks **two** weights at a time and sets them to their best values with all the others fixed, then repeats until nothing improves. Two at a time, because the weights must satisfy $\sum_i a_i y_i = 0$, so moving one alone would break that equality. The problem is convex, so there is one answer, found the same way every run, with no learning rate.

On the pumps, the weights fall into the three groups of slide 49:

- **922 pumps get weight 0**: safely on their own side, forgotten by the model;
- **268 get the ceiling,** $a_i = C = 1$: inside the slab or on the wrong side;
- **10 sit exactly on the edge**, with a weight between 0 and C.

The 268 and the 10 are the 278 support vectors. The model that is stored is those 278 points, their weights, and the intercept $b = +1.105$.

Question — who are x_i and x, during training and during prediction?

> *Slide 61: x_i and y_i are the training data, x is the point we want to classify, correct?*
>
> *It is not clear to me who x_i and x are during training.*

The prediction formula is

$$f(x) = \sum_i a_i \, y_i \, K(x_i, x) + b,$$

and in it x_i and y_i are training points and their labels (+1 faulty, −1 healthy), a_i are the learned weights, b the learned intercept, and x is the **new pump** to classify. Only the support vectors contribute, since every other a_i is zero.

During training there is no new pump yet. The objective the solver maximises (handout, section 5.5) contains only $K(x_i, x_j)$, where **both** x_i and x_j are training points. For 1,200 pumps it uses $K(x_1, x_1)$, $K(x_1, x_2)$, and so on up to $K(x_{1200}, x_{1200})$: the $m \times m$ kernel matrix. From those similarities and the labels, the solver finds the weights a_i.

So the same function K is used in two different ways:

- **Training:** $K(x_i, x_j)$, training point against training point, to find the a_i.
- **Prediction:** $K(x_i, x)$, support vector against the new point, with the a_i now fixed.

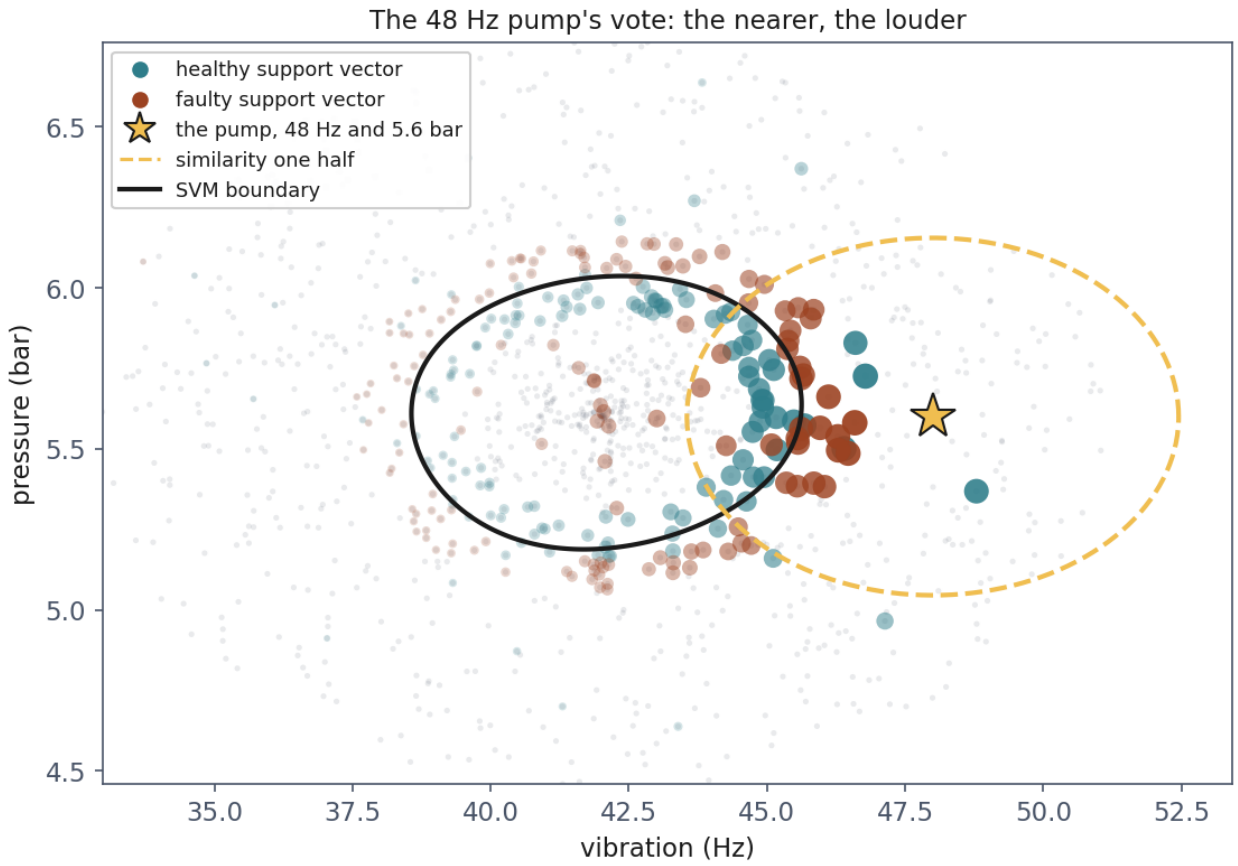
Slide 62 — The SVM on 48 Hz, 5.6 bar: +1.883 (optional)

Term	Value
votes from faulty support vectors	+37.455
votes from healthy support vectors	−36.677
intercept b	+1.105
total	**+1.883**

Prediction is the formula above, in four steps: standardise the pump with the scaler fitted on the training data; compute its RBF similarity to each of the 278 support vectors; add up weight × label × similarity, which is + for faulty support vectors and − for healthy ones; add b. The total is positive, so the pump is faulty, the same answer Naive Bayes gave with 0.981 by a completely different route. The total is exactly what `decision_function` returns, and it is a score, not a probability.

Notice that the two sides almost cancel: about +37 against about −37. The neighbourhood decides. The 56 support vectors within about 1.2 standard deviations contribute a net +1.141 towards faulty; the other 222 only −0.363. Yet no single neighbour decides: the five nearest disagree among themselves. In this sense the SVM is k-NN again, with a smooth similarity in place of a hard k, only the support vectors voting, and learned weights.

Slide 63 — One pump's vote, drawn



Each support vector sized and shaded by its similarity to the pump (the star). The 56 inside the dashed half-similarity ellipse carry most of the weight; teal and rust mix right beside the star, which is why the nearest votes nearly cancel and the neighbourhood's balance decides.

The previous table as a picture. Each support vector is sized and shaded by its similarity to the pump (the star). The dashed ellipse is where similarity falls to one half. Beside the star, teal and rust pumps lie together, pumps just outside the envelope and some flipped labels, which is why the nearest votes nearly cancel and the balance of the whole neighbourhood decides.

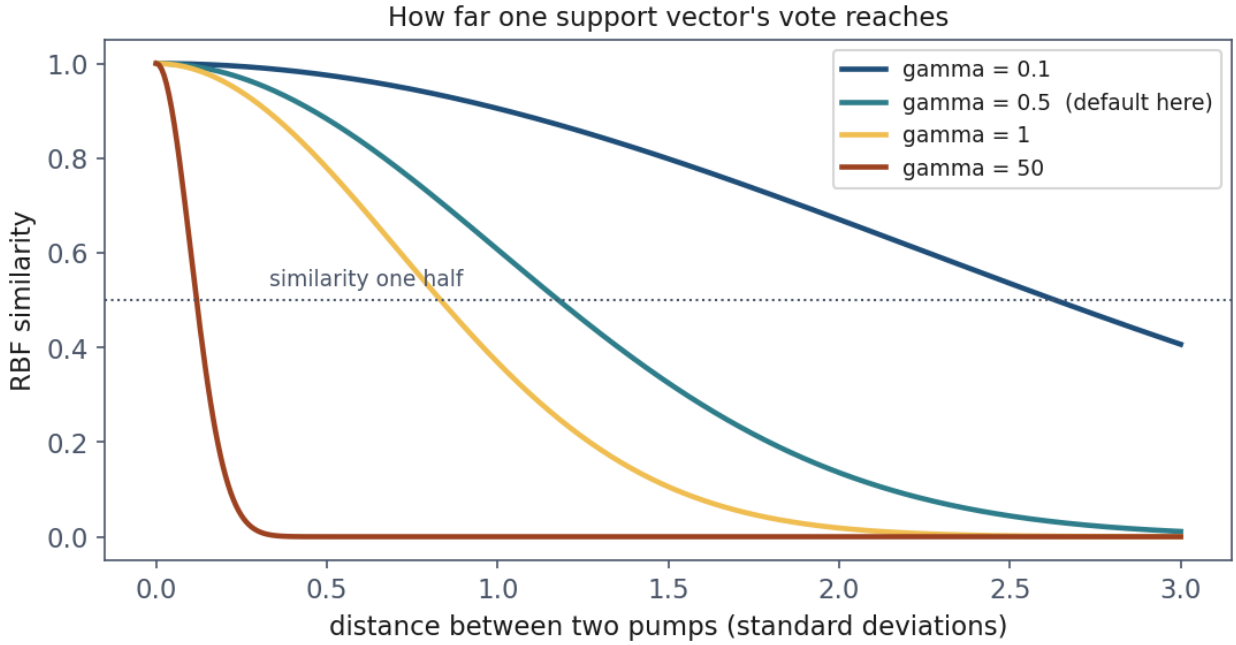
Slide 64 — Training against cross-validated, three settings (optional)

Setting	Training	Cross-validated
$= 0.1$, C $= 1$	0.936	0.929
$= 1$, C $= 1$	0.950	**0.944**
$= 50$, C $= 1000$	**0.995**	0.902

The bottom row has the best training score on the slide and the worst honest one: the signature of overfitting, measured on the same data by two procedures. Choosing on the training score would select the worst model. The instinct behind that choice is not foolish: everywhere else in engineering,

doing better on the data you have is evidence of a better system, and lessons 3 and 4 spent their time fitting data well. What breaks the analogy is that a flexible model can fit data it has already seen without learning anything that transfers.

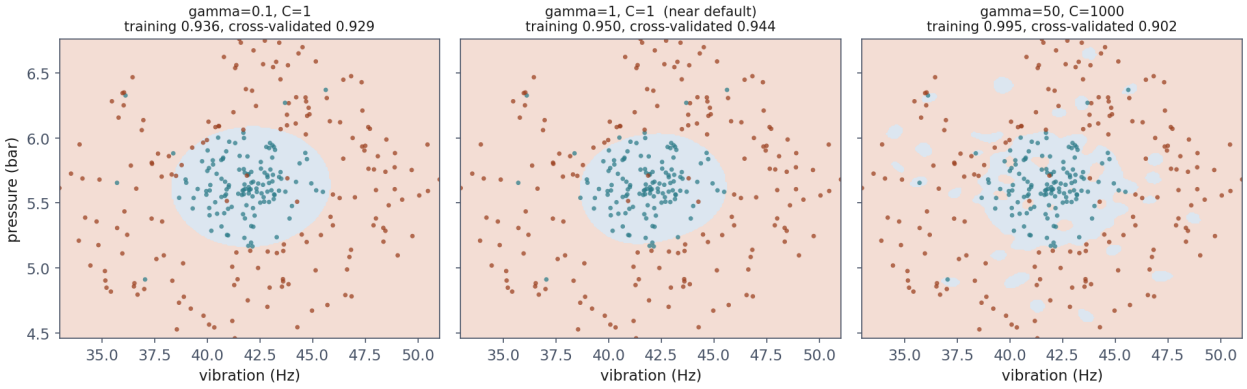
Slide 65 — How far one support vector's vote reaches, for four values of



RBF similarity against distance, one curve per γ. Where each crosses the dotted line at one half is its reach: 2.63, 1.18, 0.83 and 0.12 standard deviations.

γ turned into a distance. Where each curve crosses one half is the distance at which a support vector's vote has halved: 2.63 standard deviations at $\gamma = 0.1$, 1.18 at the default 0.5, 0.83 at 1, and 0.12 at 50. At $\gamma = 50$ each point speaks only for a neighbourhood about a tenth of a standard deviation wide. Before looking at the next slide, predict what the boundary will look like.

Slide 66 — Overfitting you can see: the boundary at the three settings



Three settings, with the training and cross-validated scores in each title. Read them together, as lesson 5 taught.

Left, $\gamma = 0.1$: a smooth boundary, slightly too smooth, and the two scores agree. Middle, $\gamma = 1$: close to the true envelope, and the best honest score. Right, $\gamma = 50$ with $C = 1000$: the boundary has broken into bubbles, small islands drawn around individual points, mislabelled ones included. That is what a training score of 0.995 looks like from outside: a private territory around each flipped label. A new pump landing between two bubbles is where the 0.902 comes from. Since lesson 5 overfitting has been a curve on a plot; here it is a shape in the input space, visibly wrong. When you can plot the boundary, plot it.

Part VI — Closing

Slide 67 — Notebook 3, live

The slide's table maps each section of the notebook to its slides and to the scikit-learn calls it uses. The notebook fits the linear and RBF SVMs, counts their support vectors, takes one pump through the vote, and sweeps γ and C with the boundary drawn each time. The step worth doing yourself is turning γ up and watching the bubbles form: it is the most direct experience of overfitting in the course. Print the support-vector fraction for the two kernels side by side, so that the diagnostic of slide 54 becomes two numbers you produced.

Slide 68 — The three families compared, on the same pumps and the same folds

Model	Accuracy
Majority baseline	0.613
Logistic regression (lesson 4)	0.613
SVM, linear kernel	0.613
Gaussian Naive Bayes	0.933
k-NN, k = 5	0.944
SVM, RBF kernel	**0.947**
noise ceiling	*0.96*

The top three rows are all linear and all exactly at the base rate. The bottom three are all near the ceiling, reached by three unrelated routes: remembering the neighbourhood, assuming independence, bending the space. The gap between the best and the worst is 0.334, larger than any difference this course has shown between a default model and a carefully tuned one; lesson 5's hyperparameter search moved scores by a few hundredths. **Choosing the right family matters far more than tuning the wrong one**, and the way to know which family you need is to look at the data first.

Slide 69 — How to choose, in practice

Situation	Reach for
Few features, plenty of data, odd-shaped boundary	k-NN, or an RBF SVM
Very many features, little data	Naive Bayes
Need a calibrated probability	**Not** Naive Bayes
Model must be small, or fast at prediction	SVM, not k-NN
The signal is an interaction between features	k-NN or a kernel

Each row is a conclusion from today's evidence rather than general advice. k-NN and the RBF SVM follow the odd-shaped pump boundary; Naive Bayes estimates one small number per feature, which is what very many features and little data require; its probabilities were uninformative on slide 45; the SVM keeps 278 points where k-NN keeps all 1,200; and on the interacting sensors k-NN and the RBF SVM scored 0.967 and 0.972 where Naive Bayes scored 0.404. One case none of the three handles: when a decision must be explained to the person it affects, a support vector machine in a lifted space offers no explanation anyone would accept.

Slide 70 — What to take away

- **No line separates the pumps**: every linear model sits at 0.613, the base rate.
- k **is the bias-variance dial**, visible in the boundaries of slide 14.
- **In 100 dimensions the nearest point is 70% as far as the farthest**, and every method built on distance suffers.
- **Naive Bayes** reaches 0.933 where breaking its assumption is cheap, and 0.404 where the signal is an interaction.
- **Choosing the family beats tuning the wrong one**: from 0.613 to 0.947.

The habit to keep is a pair: before reaching for a model, look at the shape of the data; after fitting one, check whether the assumption it depends on holds on your data, and check it properly. The within-class correlation of -0.006 said the assumption held; the squared distances said it did not; and the ellipse the model drew explained the 0.933. And one instinct to correct: more features is not free. It is nearly free for a linear model and expensive for anything built on a distance.

Slide 71 — Homework

Exercise 6 gives you a dataset and asks you to compare the three families and defend the one you choose. It is discussed at the start of the lesson on Friday 20 November. What matters in the exam is the reasoning in your notebook, not the accuracy: a well-argued choice that scores slightly lower beats a lucky winner with no justification. The argument must refer to the data, in the way the squared distances and the fitted ellipse explained Naive Bayes on the pumps. Cross-validate everything and report a spread, as lesson 5 set out, and keep the scaler inside the pipeline.